✨ React Revision Course — Part 6: Modern React & Extras

> **Topics 41–46** · Target: ~10 pages · Simple English · Revise from zero
>
> **How to use this file:** Read the *explanation*, run the *code*, then cover the page and say the *key points* out loud. If you can explain 7 out of 10 points, you've absorbed the topic.

📑 Contents

#	Topic	#	Topic
41	React 19 Features	44	Testing
42	Server Components	45	Deployment
43	TypeScript with React	46	Interview Questions
			Cheat Sheet (end)

4️⃣1️⃣ React 19 Features

> **Cleaner forms, built-in async handling, and less boilerplate.**

📖 Explanation

React 19 (released December 2024) focuses on **forms and async work**. It adds **Actions**, new hooks (`useActionState`, `useFormStatus`, `useOptimistic`), a new `use` API, and removes some old boilerplate (`forwardRef`, `Context.Provider`).

🧰 What's New at a Glance

Feature	What it does
Actions	Async functions that handle submits/updates, with automatic pending state
`<form action={fn}>`	Pass a function directly to a form; it receives `FormData`
`useActionState`	State + pending flag for a form action
`useFormStatus`	A child component reads its parent form's pending status
`useOptimistic`	Show the expected result instantly, before the server replies
`use`	Read a **Promise** or **Context** during render
`ref` **as a prop**	No more `forwardRef`
`<Context value>`	No more `.Provider`
Metadata tags	`<title>`, `<meta>` can be rendered inside components

💻 Code — Form Actions + `useActionState`

```
import { useActionState } from "react";

// The action receives (previousState, formData) and returns the NEW state
async function subscribe(prevState, formData) {
  const email = formData.get("email");              // read by input's name attribute
  await new Promise((r) => setTimeout(r, 1000));    // pretend API call
  if (!email.includes("@")) return { error: "Invalid email" };
  return { success: `Subscribed ${email}!` };
}

function SubscribeForm() {
  const [state, formAction, isPending] = useActionState(subscribe, {});

  return (
    <form action={formAction}>                      {/* function, not onSubmit */}
      <input type="email" name="email" placeholder="Email" />
      <button disabled={isPending}>{isPending ? "Saving..." : "Subscribe"}</button>
      {state.error && <p style={{ color: "red" }}>{state.error}</p>}
      {state.success && <p style={{ color: "green" }}>{state.success}</p>}
    </form>
```

```
  );
}
```

> No `e.preventDefault()`, no manual `useState` for loading, and uncontrolled form fields are **reset automatically** after a successful action.

💻 Code — `useFormStatus` (reusable submit button)

```
import { useFormStatus } from "react-dom";          // note: react-dom, not react

function SubmitButton() {
  const { pending } = useFormStatus();              // must be rendered INSIDE a <form>
  return <button disabled={pending}>{pending ? "Sending..." : "Send"}</button>;
}
```

💻 Code — `useOptimistic` (instant UI)

```
import { useOptimistic, useState } from "react";

function Messages() {
  const [messages, setMessages] = useState([]);
  const [optimistic, addOptimistic] = useOptimistic(
    messages,
    (current, newText) => [...current, { text: newText, sending: true }]
  );

  async function send(formData) {
    const text = formData.get("text");
    addOptimistic(text);                            // appears immediately (grayed out)
    const saved = await api.post("/messages", { text });   // real request
    setMessages((m) => [...m, saved.data]);         // real data replaces the temporary one
  }

  return (
    <>
      <ul>{optimistic.map((m, i) => <li key={i} style={{ opacity: m.sending ? 0.5 : 1 }}>{m.text}</li>)}</ul>
      <form action={send}><input name="text" /><button>Send</button></form>
    </>
  );
}
```

💻 Code — `use` API

```
import { use, Suspense } from "react";

function Comments({ commentsPromise }) {
```

```
  const comments = use(commentsPromise);          // suspends until the promise resolves
  return <ul>{comments.map((c) => <li key={c.id}>{c.text}</li>)}</ul>;
}

<Suspense fallback={<p>Loading comments...</p>}>
  <Comments commentsPromise={fetchComments()} />    {/* promise should be created outside render
</Suspense>

// `use` can also read context, and (unlike hooks) can be called inside if / loops:
const theme = use(ThemeContext);
```

💻 Code — Smaller Changes

```
// ref as a normal prop (no forwardRef)
function MyInput({ ref, ...props }) { return <input ref={ref} {...props} />; }

// Context without .Provider
<ThemeContext value="dark">...</ThemeContext>

// Metadata inside components (React moves it into <head>)
function BlogPost() {
  return (
    <article>
      <title>My Post</title>
      <meta name="description" content="..." />
    </article>
  );
}
```

✅ Key Points

- React 19 improves **forms and async data**, and reduces boilerplate.
- **Actions** = async functions used for mutations; React manages **pending, errors, and optimistic updates**.
- `<form action={fn}>`: the function receives `FormData`; read values with `formData.get("name")`.
- `useActionState(action, initialState)` returns `[state, formAction, isPending]`.
- `useFormStatus()` (from `react-dom`) gives a child its form's `pending` status; it must be inside the `<form>`.
- `useOptimistic` shows the **expected result immediately**, then reconciles with the real server result.
- `use(promise)` reads a Promise during render and works with `Suspense`; `use(Context)` reads context.
- Unlike hooks, `use` **can be called conditionally**.
- `ref` **is a normal prop** in function components; `forwardRef` is no longer needed.
- `<Context value={...}>` works directly; `.Provider` is no longer required.

- `<title>`, `<meta>`, `<link>` can be rendered **inside components** and React hoists them to `<head>`.
- The separate **React Compiler** can auto-memoize components, reducing manual `useMemo` / `useCallback`.
- `useActionState` was previously called `useFormState`.

> 💡 **Memory tip:** React 19 = "forms that handle themselves": pass a function to `action`, and React tracks pending, errors, and results for you.
>
> 🎤 **Say it in one line:** "React 19 adds Actions and hooks like useActionState, useOptimistic and use to simplify forms and async UI."

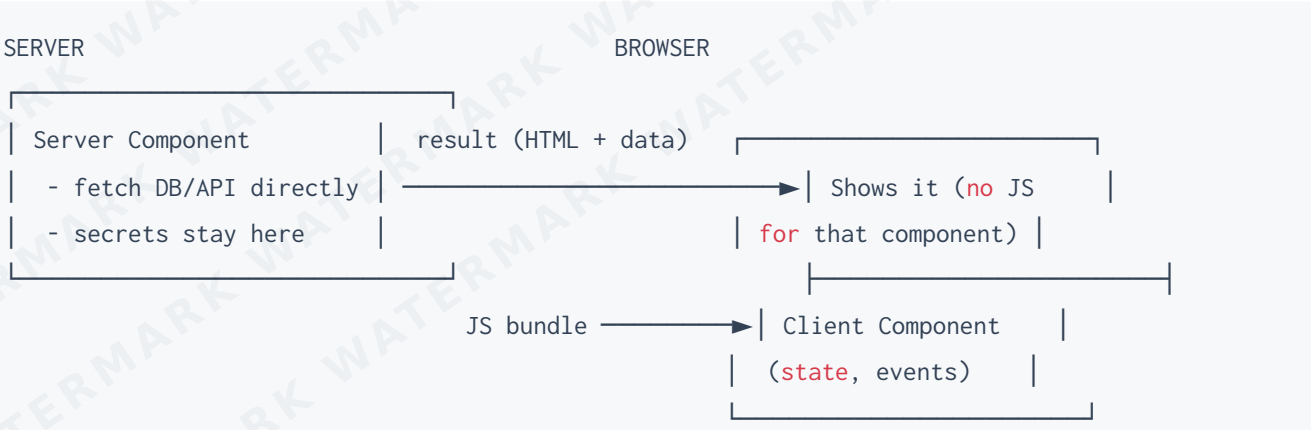
4️⃣2️⃣ Server Components

> **Components that run only on the server and send finished HTML/data to the browser, with zero JavaScript for them.**

📖 Explanation

In a normal React SPA, **all components download as JavaScript and run in the browser**. With **React Server Components (RSC)**, some components run **only on the server**: they can read the database directly and **never ship their code** to the client. You need a **framework** that supports them (for example **Next.js App Router**).

🔄 Picture



💻 Code (Next.js App Router style)

```
// app/posts/page.jsx — SERVER component by default (no directive needed)
import db from "@/lib/db";
import LikeButton from "./LikeButton";
```

```jsx
export default async function PostsPage() {          // can be async!
  const posts = await db.post.findMany();            // direct DB access, no API needed

  return (
    <ul>
      {posts.map((p) => (
        <li key={p.id}>
          {p.title}
          <LikeButton postId={p.id} />               {/* a Client Component inside */}
        </li>
      ))}
    </ul>
  );
}
```

```jsx
// app/posts/LikeButton.jsx — CLIENT component
"use client";                                        // directive at the TOP of the file

import { useState } from "react";

export default function LikeButton({ postId }) {
  const [likes, setLikes] = useState(0);             // state & events need a Client Component
  return <button onClick={() => setLikes(likes + 1)}>❤️ {likes}</button>;
}
```

```jsx
// Server Action: a function that runs on the server, callable from a form
"use server";
export async function createPost(formData) {
  await db.post.create({ data: { title: formData.get("title") } });
}
// <form action={createPost}> ... </form>
```

🧠 Server vs Client Components

	Server Component	Client Component
Directive	None (default in Next.js App Router)	`"use client"` at the top
Runs on	Server only	Browser (and pre-rendered on the server)
Can use `useState`, `useEffect`	❌ No	✅ Yes

	Server Component	Client Component
Can use event handlers (`onClick`)	❌ No	✅ Yes
Can access DB / secrets directly	✅ Yes	❌ No
JS sent to browser	❌ None	✅ Yes
Can be `async`	✅ Yes	❌ No

✅ Key Points

- **Server Components** render on the **server**; their code is **never sent to the browser**.
- Benefits: **smaller bundles**, **faster loads**, **direct DB/API access**, and **secrets stay on the server**.
- They **cannot** use state, effects, or browser-only APIs (`window`, `localStorage`) or event handlers.
- **Client Components** are marked with `"use client"` at the top and behave like the React you already know.
- In frameworks like **Next.js App Router**, components are **Server Components by default**.
- Server Components can be `async` and `await` data directly.
- A **Server Component can import and render Client Components**; a **Client Component cannot import a Server Component**, but it can receive one as `children`.
- `"use client"` marks a **boundary**: everything imported from that file also becomes client code.
- Push `"use client"` **as far down the tree as possible** (small interactive leaves like buttons).
- **Server Actions** (`"use server"`) are server functions you can call from forms or client code.
- RSC needs **framework support**; a plain Vite SPA doesn't have them.
- RSC is **not** the same as **SSR**: SSR sends initial HTML but still hydrates all components with JS; RSC components never hydrate.
- Props passed from Server to Client must be **serializable** (no functions or class instances).

> 💡 **Memory tip:** Server Components are a restaurant kitchen that serves the finished dish; Client Components are the DIY grill the customer operates at the table.
>
> 🎤 **Say it in one line:** "Server Components run only on the server and send results to the browser, reducing JavaScript and allowing direct data access."

4️⃣3️⃣ TypeScript with React

> **Add types to props, state, events and hooks to catch bugs before running the app.**

📖 Explanation

TypeScript is JavaScript with **types**. In React it checks that you pass the right props, use state correctly and handle events safely. Files use `.tsx` (TypeScript + JSX).

💻 Create a TS Project

```
cd C:\Users\YourName\Projects
npm create vite@latest my-ts-app -- --template react-ts
```

💻 Typing Props

```
interface ButtonProps {
  label: string;                        // required
  size?: "sm" | "md" | "lg";            // optional + limited to exact values (union)
  disabled?: boolean;
  onClick: (id: number) => void;        // function prop with its signature
  children?: React.ReactNode;           // anything renderable
}

function Button({ label, size = "md", disabled = false, onClick, children }: ButtonProps) {
  return (
    <button className={size} disabled={disabled} onClick={() => onClick(1)}>
      {label} {children}
    </button>
  );
}

// <Button label="Save" onClick={(id) => console.log(id)} />   ✅
// <Button label={5} />                                        ❌ TS error: number is not a str
```

💻 Typing State

```
const [count, setCount] = useState(0);                    // inferred: number
const [name, setName] = useState<string>("");             // explicit
const [user, setUser] = useState<User | null>(null);      // starts null, later a User
const [items, setItems] = useState<string[]>([]);         // empty array NEEDS a type
```

```ts
interface User { id: number; name: string; email: string; }
// user?.name  ← optional chaining because it may be null
```

💻 Typing Events & Refs

```tsx
function Form() {
  const inputRef = useRef<HTMLInputElement>(null);          // DOM ref: element type + null

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    console.log(e.target.value);
  };
  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    inputRef.current?.focus();                              // current can be null → use ?.
  };
  const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => {};

  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} onChange={handleChange} />
      <button onClick={handleClick}>Go</button>
    </form>
  );
}
```

💻 Typing Reducer, Context & Custom Hooks

```ts
// useReducer with a discriminated union of actions
type State = { count: number };
type Action = { type: "inc" } | { type: "dec" } | { type: "add"; payload: number };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "inc": return { count: state.count + 1 };
    case "dec": return { count: state.count - 1 };
    case "add": return { count: state.count + action.payload };   // TS knows payload exists here
  }
}

// Context
interface AuthContextType { user: User | null; login: (u: User) => void; logout: () => void; }
const AuthContext = createContext<AuthContextType | null>(null);

// Generic custom hook
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  // ...
  return { data };
```

```
}
const { data } = useFetch<User[]>("/api/users");              // data is User[] | null
```

✅ Key Points

- Use `.tsx` for files containing JSX; create a project with `--template react-ts`.
- Type props with an `interface` or `type`: `function Card({ title }: CardProps)`.
- `?` marks an **optional** prop; give a **default** in destructuring.
- **Union types** limit values: `"sm" | "md" | "lg"`.
- `React.ReactNode` is the type for `children` (anything renderable).
- Type function props with their signature: `(id: number) => void`.
- `useState<Type>()`: TS infers simple types; add a generic for `null` **unions** and **empty arrays**.
- Event types: `React.ChangeEvent<HTMLInputElement>`, `React.FormEvent<HTMLFormElement>`, `React.MouseEvent<HTMLButtonElement>`.
- `useRef<HTMLInputElement>(null)`; use `ref.current?.` because it can be `null`.
- Type reducer **actions as a union** with a `type` field so TS narrows each `case`.
- Create context with `createContext<Type | null>(null)`, and check for `null` in your `useX` hook.
- **Generics** make reusable hooks/components type-safe (`useFetch<User[]>`).
- Avoid `any`; it turns off type checking. Use `unknown` when you truly don't know.
- Define **API response types** (`interface User`) and reuse them across the app.
- Use your editor's **hover** and **error messages** to discover correct event/element types.
- `interface` **vs** `type`: both work for props; `interface` is common for object shapes, `type` for unions.

> 💡 **Memory tip:** TypeScript is a spell-checker for your code: it underlines the mistake before you run it.
>
> 🎤 **Say it in one line:** "TypeScript adds static types to props, state, events and hooks so errors are caught at compile time."

4️⃣4️⃣ Testing

> **Write automated tests that check your components behave the way a user expects.**

📖 Explanation

React Testing Library (RTL) tests components **the way users use them**: find elements by what's visible (text, role, label), click, type, and check the result. It runs inside a test runner: **Vitest** (best with Vite) or **Jest** (same ideas, nearly identical syntax).

🧰 The Tools

Tool	Role
Vitest / Jest	Test runner: `describe`, `it/test`, `expect`, mocks
React Testing Library	`render`, `screen`, queries
user-event	Simulates real user actions (type, click)
jest-dom	Extra matchers: `toBeInTheDocument()`, `toBeDisabled()`
jsdom	Fake browser environment in Node

💻 Setup (Vite + Vitest)

```
cd C:\Users\YourName\Projects\my-app
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/use
```

```
// vite.config.js
export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    globals: true,                    // use describe/it/expect without importing
    setupFiles: "./src/setupTests.js",
  },
});
```

```
// src/setupTests.js
import "@testing-library/jest-dom";
```

```
// package.json → "scripts": { "test": "vitest" }
```

```
cd C:\Users\YourName\Projects\my-app
npm test
```

💻 Test 1 — A Simple Component

```
// Counter.test.jsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
```

```
import Counter from "./Counter";

describe("Counter", () => {
  it("starts at 0", () => {
    render(<Counter />);
    expect(screen.getByText(/count: 0/i)).toBeInTheDocument();
  });

  it("increments when the button is clicked", async () => {
    const user = userEvent.setup();
    render(<Counter />);

    await user.click(screen.getByRole("button", { name: /increment/i }));

    expect(screen.getByText(/count: 1/i)).toBeInTheDocument();
  });
});
```

💻 Test 2 — Forms, Mock Functions & Async Data

```
import { vi } from "vitest";        // (Jest: use `jest.fn()` instead of `vi.fn()`)

it("submits the form with typed values", async () => {
  const user = userEvent.setup();
  const onSubmit = vi.fn();                              // mock function
  render(<LoginForm onSubmit={onSubmit} />);

  await user.type(screen.getByLabelText(/email/i), "ali@mail.com");
  await user.type(screen.getByLabelText(/password/i), "secret123");
  await user.click(screen.getByRole("button", { name: /login/i }));

  expect(onSubmit).toHaveBeenCalledWith({ email: "ali@mail.com", password: "secret123" });
});

it("shows users after loading", async () => {
  vi.spyOn(global, "fetch").mockResolvedValue({
    ok: true,
    json: async () => [{ id: 1, name: "Sara" }],
  });

  render(<Users />);

  expect(screen.getByText(/loading/i)).toBeInTheDocument();
  expect(await screen.findByText("Sara")).toBeInTheDocument();   // findBy waits for async conter
});
```

🧠 Query Types

Prefix	Not found	Found multiple	Use for
`getBy…`	❌ throws	❌ throws	Element **must exist** now
`queryBy…`	returns `null`	❌ throws	Assert element **does NOT exist**
`findBy…`	❌ rejects (after waiting)	❌ rejects	Element appears **later** (async)
`getAllBy…`	❌ throws	✅ array	Several elements

Query priority (best → worst): `getByRole` → `getByLabelText` → `getByPlaceholderText` → `getByText` → `getByTestId` (last resort).

✅ Key Points

- RTL tests **behavior as the user sees it**, not internal state or implementation.
- `render(<Component />)` mounts it; `screen` queries the rendered output.
- Prefer `getByRole` (buttons, headings, textboxes); it also checks accessibility.
- `getBy` = must exist; `queryBy` = may be absent (returns `null`); `findBy` = waits for async appearance.
- Use `userEvent` (`await user.click/type`) rather than `fireEvent`; it simulates real interactions.
- `await` async actions and `findBy` queries.
- **jest-dom matchers**: `toBeInTheDocument`, `toBeDisabled`, `toHaveValue`, `toHaveTextContent`.
- **Mock functions** (`vi.fn()` / `jest.fn()`) verify callbacks were called with the right arguments.
- **Mock network calls** (mock `fetch`/Axios, or use **MSW**) so tests never hit real APIs.
- Test names should read like sentences: *"it shows an error when the email is invalid"*.
- Structure each test as **Arrange** → **Act** → **Assert**.
- Don't test **implementation details** (state variable names, internal functions).
- Tests must be **independent**; clean up happens automatically after each test in RTL.
- **Vitest and Jest** share almost the same API; Vitest is faster and works natively with Vite.
- Also know: **unit tests** (one function/component), **integration tests** (several parts together), **E2E tests** (whole app in a real browser, e.g., Playwright/Cypress).

> 💡 **Memory tip:** Test like a user, not like a developer: "I click Login and expect to see Welcome", not "I expect state.loggedIn to be true".
>
> 🎤 **Say it in one line:** "I use React Testing Library with Vitest or Jest to test components from the user's perspective."

4️⃣5️⃣ Deployment

> **Build your app and put it on the internet.**

📖 Explanation

A Vite React app is just **static files** (HTML, CSS, JS). `npm run build` bundles and minifies everything into the `dist/` folder, which you can host on any static host: **Vercel**, **Netlify**, GitHub Pages, etc.

🔄 The Process

```
Write code → npm run build → dist/ folder → upload/connect to host → live URL
                                (minified, optimized static files)
```

💻 Build & Preview Locally

```
cd C:\Users\YourName\Projects\my-app
npm run build
```

```
cd C:\Users\YourName\Projects\my-app
npm run preview
```

💻 Deploy on Vercel (easiest, via GitHub)

1. Push your project to GitHub.
2. Go to vercel.com → "Add New Project" → import your repo.
3. Framework Preset: Vite (auto-detected). Build command: npm run build. Output: dist
4. Add Environment Variables (e.g. VITE_*API*_URL=https://your-api.onrender.com/api)
5. Click Deploy → every `git push` redeploys automatically.

💻 Deploy on Netlify

1. Push to GitHub → netlify.com → "Add new site" → import from Git.
2. Build command: npm run build Publish directory: dist
3. Add environment variables in Site settings.

🚨 Fix "404 on Refresh" (very common with React Router)

A SPA has only one real file (`index.html`). Visiting `/about` directly makes the server look for `/about` , which doesn't exist → 404. Tell the host to **always serve** `index.html` :

```
# Netlify: create  public/_redirects
/*    /index.html   200
```

```
// Vercel: create  vercel.json  in the project root
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

💻 Environment Variables

```
# .env.development          → used by `npm run dev`
VITE_API_URL=http://localhost:5000/api

# .env.production           → used by `npm run build`
VITE_API_URL=https://my-api.onrender.com/api
```

```
const API = import.meta.env.VITE_API_URL;     // value is baked in at BUILD time
```

✅ Pre-Deploy Checklist

✔	Check
☐	`npm run build` succeeds with **no errors**
☐	`npm run preview` works locally
☐	**No hard-coded** `localhost` URLs
☐	Env variables set on the host (prefixed `VITE_`)
☐	SPA rewrite/redirect added (`_redirects` or `vercel.json`)
☐	Backend deployed and **CORS** `origin` set to the live frontend URL
☐	`.env` is in `.gitignore` ; **no secrets** in frontend code
☐	Remove stray `console.log` s and test data

✅ Key Points

- `npm run build` creates an optimized, minified `dist/` folder (static files).

- `npm run preview` serves the production build locally for a final check.
- Static hosts: **Vercel, Netlify, GitHub Pages, Cloudflare Pages**.
- Easiest workflow: **connect a GitHub repo**, and every push **auto-deploys**.
- Vite's settings are: **build command** `npm run build`, **output directory** `dist`.
- **React Router needs a rewrite rule** so refreshing a deep link serves `index.html` (`_redirects` / `vercel.json`).
- Env variables must start with `VITE_`, are **embedded at build time**, and are **visible to anyone**: never put secrets in them.
- Changing an env variable on the host requires a **rebuild/redeploy**.
- The **backend deploys separately** (Render, Railway, etc.); update the frontend's `VITE_API_URL` and the backend's **CORS origin**.
- On **GitHub Pages** (a subpath URL), set `base` in `vite.config.js` and use `HashRouter` or a 404 workaround.
- Use **HTTPS** (hosts provide it free) since cookies, tokens and many browser APIs require it.
- Test the live site: check the **Network tab** for failed requests or CORS errors.

> 💡 **Memory tip:** Build = pack your suitcase (`dist`); deploy = carry it to the airport (host); the rewrite rule = telling every gate to send passengers to the main terminal (`index.html`).
>
> 🎤 **Say it in one line:** "I run npm run build to produce static files in dist, deploy them to a host like Vercel or Netlify, and add an SPA rewrite for client-side routing."

4️⃣6️⃣ Interview Questions

> **30 questions with short, ready-to-say answers.**

🟢 Basics

#	Question	Answer
1	**What is React?**	A JavaScript library for building user interfaces from reusable components, using a virtual DOM for efficient updates.
2	**What is the virtual DOM?**	A lightweight JS copy of the real DOM. React diffs old vs new versions and updates only what changed.
3	**What is JSX?**	HTML-like syntax inside JavaScript that compiles to `React.createElement` calls.

#	Question	Answer
4	**Props vs state?**	Props are read-only inputs from a parent; state is data owned by a component that triggers re-renders when changed.
5	**Why are keys needed in lists?**	So React can identify which items changed, moved, or were removed, for correct and efficient updates.
6	**Controlled vs uncontrolled components?**	Controlled: the input's value lives in React state. Uncontrolled: the DOM holds the value, read with a ref.
7	**What is lifting state up?**	Moving state to the closest common parent so sibling components can share it.
8	**Why must components start with a capital letter?**	Lowercase names are treated as HTML tags; capitalized names are treated as components.
9	**What is a Fragment?**	`<>...</>` lets you return multiple elements without adding an extra DOM node.
10	**Why `className` instead of `class`?**	`class` is a reserved word in JavaScript.

🟡 Hooks

#	Question	Answer
11	**What are the Rules of Hooks?**	Call hooks only at the top level (not in loops/conditions) and only from React functions or custom hooks.
12	**Explain `useEffect` and its dependency array.**	Runs side effects after render. No array = every render; `[]` = once; `[x]` = when `x` changes. Return a function for cleanup.
13	**`useState` vs `useRef`?**	State changes trigger re-renders; ref changes don't. Refs also hold DOM nodes.
14	**`useMemo` vs `useCallback`?**	`useMemo` caches a computed value; `useCallback` caches a function reference.

#	Question	Answer
15	**When to use `useReducer`?**	When state is complex or many actions update related state; it centralizes update logic.
16	**What is a custom hook?**	A function starting with `use` that calls other hooks to reuse stateful logic (state isn't shared between callers).
17	**Why does `useEffect` run twice in development?**	StrictMode mounts, unmounts and re-mounts to verify your cleanup logic works.
18	**What is prop drilling and how do you avoid it?**	Passing props through many layers that don't use them. Use Context or a state library.

🟠 Advanced

#	Question	Answer
19	**How does `React.memo` work?**	It skips re-rendering when props are shallow-equal; use stable props via `useMemo` / `useCallback`.
20	**What are error boundaries?**	Class components that catch render errors in their children and show a fallback UI; they don't catch event/async errors.
21	**What is code splitting?**	Loading code on demand with `React.lazy` and `Suspense` to shrink the initial bundle.
22	**What is a portal?**	`createPortal` renders children into a different DOM node while staying in the same React tree (modals, tooltips).
23	**HOC vs render props vs hooks?**	HOCs wrap components, render props pass a function prop; both nest deeply. Hooks do the same job more simply.
24	**How do you optimize performance?**	Profile first; keep state local, use `memo` / `useMemo` / `useCallback`, virtualize big lists, code split, and debounce events.
25	**What is reconciliation?**	The process of diffing the new virtual DOM against the old one to compute minimal DOM updates.

🔴 Ecosystem & Modern

#	Question	Answer
26	**Context vs Redux?**	Context is built in and good for low-frequency data (theme, auth); Redux is for large, complex, frequently updated global state, with better tooling.
27	**What is server state and why TanStack Query?**	Data owned by the server that can go stale; the library handles caching, refetching and loading/error states.
28	**How does JWT authentication work?**	Server signs a token at login; the client sends it in the `Authorization` header; the server verifies it on each request.
29	**What's new in React 19?**	Actions, `useActionState`, `useFormStatus`, `useOptimistic`, the `use` API, `ref` as a prop, and simpler Context syntax.
30	**Server vs Client Components?**	Server Components run only on the server (no JS shipped, direct data access); Client Components (`"use client"`) handle state and events in the browser.

🎯 How to Answer Any Interview Question

1. **Define it** in one line.
2. **Say why it exists** (the problem it solves).
3. **Give a tiny example** (from this PDF's code).
4. **Mention one gotcha** (shows depth).

📝 Part 6 Cheat Sheet

Topic	Remember this
React 19	Actions, `useActionState`, `useFormStatus`, `useOptimistic`, `use`, `ref` as prop, `<Context value>`
Server Components	Run on server only, no JS shipped; `"use client"` for state/events; need a framework (Next.js)
TypeScript	Type props (`interface`), `useState<T>`, event types, `useRef<HTMLX>(null)`, generics, avoid `any`

Topic	Remember this
Testing	Vitest/Jest + RTL; `render`, `screen.getByRole`, `userEvent`, `findBy` for async, mock fetch
Deployment	`npm run build` → `dist/`; Vercel/Netlify; SPA rewrite; `VITE_` env vars; CORS origin
Interviews	Define → why → example → gotcha

🔥 Top 6 Quick-Fire Questions

1. **What does `useActionState` return?** → `[state, formAction, isPending]`.
2. **What does `"use client"` do?** → Marks a file as a Client Component boundary (state, effects, events allowed).
3. **How do you type a ref to an input?** → `useRef<HTMLInputElement>(null)`.
4. **`getBy` vs `queryBy` vs `findBy`?** → Must exist now / may not exist (returns `null`) / appears later (async).
5. **Why does a refresh on `/about` give 404 after deploying?** → The host looks for a real `/about` file; add an SPA rewrite to `index.html`.
6. **Where do secrets belong?** → On the server, never in frontend code or `VITE_` variables.

🎓 Course Complete: How to Merge Everything

You now have all 6 parts:

File	Topics
`React_Part1_Fundamentals.md`	1–13
`React_Part2_Hooks.md`	14–23
`React_Part3_Routing_Data.md`	24–27
`React_Part4_Performance_Quality.md`	28–34
`React_Part5_State_Ecosystem.md`	35–40

File	Topics
`React_Part6_Modern_Extras.md`	41–46

To make one PDF: paste all six files into one `.md` file in order (Part 1 → 6), then export to PDF using VS Code (Markdown PDF extension) or Typora.

How to revise: read one part per session → cover the page → say each topic's *key points* and *one-liner* out loud → check the *quick-fire questions* → finish with the *interview section*.

🎉 **Happy revising!**

WATERMARK WATERMARK WATERMARK

🗄️ React Revision Course — Part 5: State Management & Ecosystem

> **Topics 35–40** · Target: ~15 pages · Simple English · Revise from zero
>
> **How to use this file:** Read the *explanation*, run the *code*, then cover the page and say the *key points* out loud. If you can explain 7 out of 10 points, you've absorbed the topic.

📑 Contents

#	Topic	#	Topic
35	Redux Toolkit	38	React Hook Form
36	Zustand	39	Authentication Flow
37	TanStack Query	40	Connecting to a Backend (MERN)
			Cheat Sheet (end)

🧭 First, Pick the Right Tool

Not all "state" is the same. Choosing the right tool is half the skill.

Kind of state	Example	Best tool
Local UI state	Input text, modal open/closed	`useState` / `useReducer`
Shared UI state (small)	Theme, current user	Context
Shared client state (large/complex)	Cart, filters, wizard steps	**Redux Toolkit** or **Zustand**
Server state	Data from an API (users, posts)	**TanStack Query** (or RTK Query)

Kind of state	Example	Best tool
Form state	Fields, errors, validation	**React Hook Form**

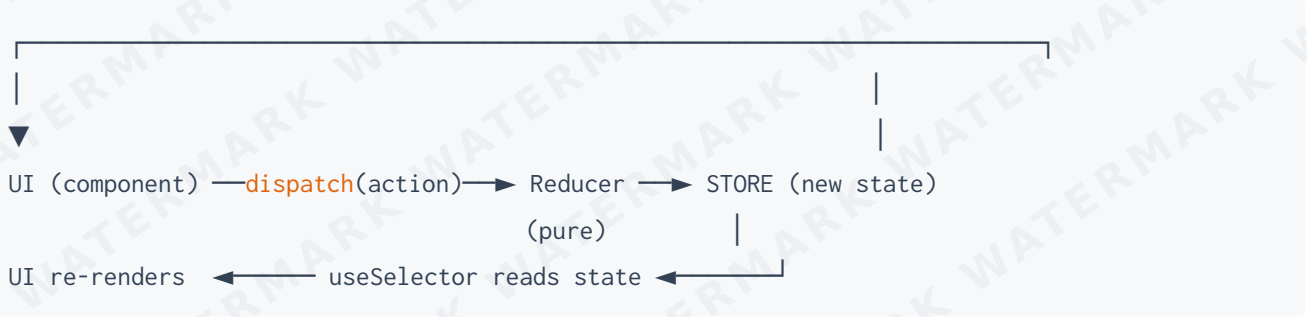
3️⃣5️⃣ Redux Toolkit

> **The official, modern way to use Redux: one global store, predictable updates, much less boilerplate.**

📖 Explanation

Redux keeps your app's shared state in **one central store**. Components **don't change it directly**; they **dispatch actions**, and **reducers** compute the new state. **Redux Toolkit (RTK)** is the recommended way to write Redux: it removes most of the old boilerplate.

🔄 The Redux Flow (memorize this)



💻 Install

```
cd C:\Users\YourName\Projects\my-app
npm install @reduxjs/toolkit react-redux
```

💻 Step 1 — Create a Slice (state + reducers + actions together)

```
// src/features/counter/counterSlice.js
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    // RTK uses Immer: you can WRITE "mutating" code, and it's turned into safe immutable updates
    increment: (state) => { state.value += 1; },
```

```
    decrement: (state) => { state.value -= 1; },
    addBy: (state, action) => { state.value += action.payload; },
    reset: (state) => { state.value = 0; },
  },
});

export const { increment, decrement, addBy, reset } = counterSlice.actions;  // action creators
export default counterSlice.reducer;                                          // the reducer
```

💻 Step 2 — Create the Store

```
// src/app/store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";
import cartReducer from "../features/cart/cartSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,     // state.counter
    cart: cartReducer,           // state.cart
  },
});   // Redux DevTools + thunk middleware are already included
```

💻 Step 3 — Provide the Store

```
// main.jsx
import { Provider } from "react-redux";
import { store } from "./app/store";

createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

💻 Step 4 — Use It in Components

```
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement, addBy } from "./features/counter/counterSlice";

function Counter() {
  const value = useSelector((state) => state.counter.value);   // READ (select only what you nee
  const dispatch = useDispatch();                              // WRITE

  return (
    <>
      <h2>{value}</h2>
```

```
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(addBy(5))}>+5</button>
    </>
  );
}
```

💻 Async Logic — `createAsyncThunk`

```
// src/features/users/usersSlice.js
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import axios from "axios";

// 1️⃣ Define the async action. It auto-generates pending / fulfilled / rejected actions.
export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async (_, { rejectWithValue }) => {
    try {
      const { data } = await axios.get("https://jsonplaceholder.typicode.com/users");
      return data;                                  // becomes action.payload on success
    } catch (err) {
      return rejectWithValue(err.message);          // becomes action.payload on failure
    }
  }
);

const usersSlice = createSlice({
  name: "users",
  initialState: { list: [], status: "idle", error: null },
  reducers: {},
  // 2️⃣ Handle the 3 lifecycle actions in extraReducers
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => { state.status = "loading"; })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.list = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.payload;
      });
  },
});
export default usersSlice.reducer;
```

```
// 3️⃣ Use it in a component
function Users() {
  const dispatch = useDispatch();
```

```
  const { list, status, error } = useSelector((s) => s.users);

  useEffect(() => {
    if (status === "idle") dispatch(fetchUsers());
  }, [status, dispatch]);

  if (status === "loading") return <p>Loading...</p>;
  if (status === "failed") return <p>{error}</p>;
  return <ul>{list.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

🧠 Redux Vocabulary

Term	Meaning
Store	The single object holding all global state
Action	Plain object describing what happened: `{ type, payload }`
Reducer	Pure function `(state, action) → newState`
Slice	One feature's state + reducers + actions (`createSlice`)
Dispatch	Function that sends an action to the store
Selector	Function that reads a piece of state: `state => state.cart.items`
Thunk	A function that performs async work and dispatches actions

✅ Key Points

- Redux = **one global store**, **one-way data flow**, **predictable** updates.
- **Redux Toolkit** is the **official** way to write Redux; plain Redux is rarely written by hand now.
- Flow: **UI → dispatch(action) → reducer → store → UI**.
- `createSlice` bundles `name`, `initialState`, `reducers` and auto-generates **action creators**.
- RTK uses **Immer**: inside reducers you can write `state.value += 1` safely (it becomes an immutable update).
- Outside `createSlice`, **never mutate state**; reducers must stay **pure** (no API calls, no random values).
- `configureStore` creates the store; DevTools and thunk middleware are **built in**.
- Wrap the app in `<Provider store={store}>` once.
- `useSelector` reads state; select **only what you need** to avoid extra re-renders.

- `useDispatch` gives you `dispatch`; call `dispatch(actionCreator(payload))`.
- `createAsyncThunk` handles async work and generates **pending / fulfilled / rejected** actions handled in `extraReducers`.
- Store only **shared, global** state; keep local UI state in `useState`.
- Don't put **non-serializable** values (functions, class instances) in the store.
- For API caching, **RTK Query** (part of RTK) or **TanStack Query** is usually better than hand-written thunks.
- Use Redux when state is **large, shared widely, and updated from many places**; otherwise Context/Zustand is simpler.

> 💡 **Memory tip:** Redux is a bank: you can't touch the vault (state). You file a slip (action), the teller (reducer) updates the balance, everyone sees the new statement.
>
> 🎤 **Say it in one line:** "Redux Toolkit manages global state in a single store using slices, actions and reducers, with predictable one-way data flow."

3️⃣6️⃣ Zustand

> **A tiny, simple global state library: no providers, no reducers, no boilerplate.**

📖 Explanation

Zustand (German for "state") lets you create a **store as a hook**. Components call the hook to read state and call functions to update it. It's much lighter than Redux and is great for small-to-medium apps.

💻 Install

```
cd C:\Users\YourName\Projects\my-app
npm install zustand
```

💻 Create and Use a Store

```
// src/store/useCartStore.js
import { create } from "zustand";

const useCartStore = create((set, get) => ({
  // STATE
  items: [],

  // ACTIONS (functions that call `set`)
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
```

```
  removeItem: (id) => set((state) => ({ items: state.items.filter((i) => i.id !== id) })),
  clear: () => set({ items: [] }),

  // Derived value using get()
  total: () => get().items.reduce((sum, i) => sum + i.price, 0),
}));

export default useCartStore;
```

```
// Any component, with NO Provider needed
import useCartStore from "./store/useCartStore";

function CartIcon() {
  const count = useCartStore((state) => state.items.length);   // selector: re-renders only when
  return <span>🛒 {count}</span>;
}

function ProductCard({ product }) {
  const addItem = useCartStore((state) => state.addItem);
  return <button onClick={() => addItem(product)}>Add to cart</button>;
}
```

💻 Async Actions & Persistence

```
import { create } from "zustand";
import { persist } from "zustand/middleware";

const useUserStore = create(
  persist(                                    // saves to localStorage automatically
    (set) => ({
      users: [],
      loading: false,
      fetchUsers: async () => {
        set({ loading: true });
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        set({ users: await res.json(), loading: false });   // async is just normal code
      },
    }),
    { name: "user-storage" }                  // localStorage key
  )
);
```

🧠 Redux Toolkit vs Zustand vs Context

	Context	Zustand	Redux Toolkit
Setup	Medium	✅ Very small	More files
Provider needed	Yes	❌ No	Yes
Re-render control	❌ All consumers	✅ Selectors	✅ Selectors
DevTools	❌ No	✅ Via middleware	✅ Built in
Best for	Theme, auth	Small/medium global state	Large apps, big teams

✅ Key Points

- Zustand = a **small global state library** built on hooks.
- `create((set, get) => ({ ... }))` returns a **hook** (convention: `useSomethingStore`).
- State and actions live **together** in one object.
- `set` updates state (it **merges** the top level); `get` reads current state.
- **No Provider** needed; import the hook and use it anywhere.
- Use a **selector** `useStore((s) => s.items)` so a component re-renders **only when that slice changes**.
- **Async actions** are just normal `async` functions that call `set`.
- Middleware: `persist` (localStorage), `devtools` (Redux DevTools), `immer`.
- Much **less boilerplate** than Redux, but less structure, so it's better for small/medium apps.
- Avoid selecting the **whole store** (`useStore()`) since it re-renders on every change.

> 💡 **Memory tip:** Zustand is a shared whiteboard in the hallway: anyone can read it or write on it, no paperwork.
>
> 🎤 **Say it in one line:** "Zustand is a minimal global store created with a hook, with no providers or reducers."

3️⃣7️⃣ TanStack Query (React Query)

> **Manage *server state*: fetching, caching, refetching and updating API data, with almost no code.**

📖 Explanation

Data from an API is **server state**: it lives on the server, can get **stale**, and is shared by many components. Manually writing `useEffect` + `loading` + `error` + caching is tedious and error-prone. **TanStack Query** does all of it for you.

🔄 What It Does For You

```
useQuery(["users"]) ──► cached? ── yes ──► show cached data instantly (+ refetch if stale)
                                └─ no ──► fetch ──► cache ──► show
```

💻 Install

```
cd C:\Users\YourName\Projects\my-app
npm install @tanstack/react-query
```

💻 Setup

```
// main.jsx
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

createRoot(document.getElementById("root")).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

💻 Reading Data — `useQuery`

```
import { useQuery } from "@tanstack/react-query";
import api from "./api";

function Users() {
  const { data, isPending, isError, error, refetch, isFetching } = useQuery({
    queryKey: ["users"],                              // unique cache key
    queryFn: async () => (await api.get("/users")).data,   // must return a Promise
    staleTime: 60 * 1000,                             // fresh for 1 minute (no refetch)
  });

  if (isPending) return <p>Loading...</p>;
  if (isError) return <p>Error: {error.message}</p>;

  return (
```

```
    <>
      {isFetching && <small>Updating in background...</small>}
      <ul>{data.map((u) => <li key={u.id}>{u.name}</li>)}</ul>
      <button onClick={() => refetch()}>Refresh</button>
    </>
  );
}

// With a parameter: the key includes the variable, so each id has its own cache
function User({ id }) {
  const { data } = useQuery({
    queryKey: ["user", id],
    queryFn: async () => (await api.get(`/users/${id}`)).data,
    enabled: !!id,                                  // don't run until id exists
  });
  // ...
}
```

💻 Changing Data — `useMutation` + Invalidate

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

function AddUser() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (newUser) => api.post("/users", newUser),
    onSuccess: () => {
      // Mark "users" as stale → it refetches automatically → list updates
      queryClient.invalidateQueries({ queryKey: ["users"] });
    },
  });

  return (
    <button
      disabled={mutation.isPending}
      onClick={() => mutation.mutate({ name: "Ali" })}
    >
      {mutation.isPending ? "Saving..." : "Add user"}
    </button>
  );
}
```

🧠 Key Options

Option	Meaning
`queryKey`	Unique ID for the cache entry; changes → new fetch
`queryFn`	Function that fetches and returns data
`staleTime`	How long data counts as *fresh* (default `0` → stale immediately)
`gcTime`	How long unused data stays in cache before deletion (default 5 min)
`enabled`	`false` pauses the query (dependent queries)
`refetchOnWindowFocus`	Refetch when the user returns to the tab (default `true`)
`retry`	Auto-retry count on failure (default 3)

✅ Key Points

- It manages **server state**: fetching, **caching**, **background refetching**, retries and syncing.
- Replaces the manual `useEffect` + `useState(loading/error/data)` pattern.
- `useQuery` is for **reading** (GET); `useMutation` is for **changing** (POST/PUT/DELETE).
- Always provide an object: `{ queryKey, queryFn }` (v5 syntax).
- `queryKey` identifies the cached data. Include **variables** (`["user", id]`) so each has its own entry.
- `queryFn` must **return a Promise** and should **throw** on errors (fetch needs a manual `res.ok` check).
- Status flags: `isPending` (no data yet), `isError`, `isFetching` (any fetch, including background).
- `staleTime`: how long data is *fresh*; `gcTime`: how long *unused* data stays cached.
- Data shows **instantly from the cache**, then refetches in the background if stale.
- After a mutation, call `queryClient.invalidateQueries({ queryKey })` to refresh related data.
- `enabled` lets you run a query only when a condition is true (dependent queries).
- Refetches automatically on **window focus**, **reconnect** and **mount** (configurable).
- Wrap the app in `QueryClientProvider` once.
- Use **React Query DevTools** (`@tanstack/react-query-devtools`) to inspect the cache.
- Use it **together** with Zustand/Redux/Context for **client state**; don't copy server data into Redux.

> 💡 **Memory tip:** TanStack Query is a smart assistant with a notebook: it remembers answers (cache), checks for updates quietly, and tells you only when something changed.

> 🎤 **Say it in one line:** "TanStack Query handles fetching, caching and syncing of server data using useQuery and useMutation."

3️⃣8️⃣ React Hook Form

> **Fast, minimal-code forms with built-in validation and fewer re-renders.**

📖 Explanation

In Part 1 you built controlled forms by hand: one state per field, one `onChange`, manual validation. For big forms it's repetitive and re-renders on every keystroke. **React Hook Form (RHF)** uses **uncontrolled inputs + refs** internally, so it's faster and needs far less code.

💻 Install

```
cd C:\Users\YourName\Projects\my-app
npm install react-hook-form
```

💻 Basic Form with Validation

```
import { useForm } from "react-hook-form";

function SignupForm() {
  const {
    register,                              // connects an input to RHF
    handleSubmit,                          // wraps your submit function (validates first)
    watch,                                 // read live values
    reset,                                 // clear the form
    formState: { errors, isSubmitting },
  } = useForm({ defaultValues: { name: "", email: "", password: "" } });

  const onSubmit = async (data) => {       // runs ONLY if validation passes
    await new Promise((r) => setTimeout(r, 1000));   // fake API call
    console.log(data);                     // { name, email, password }
    reset();
  };

  const password = watch("password");

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      <input
        placeholder="Name"
        {...register("name", { required: "Name is required", minLength: { value: 3, message: "Mi
```

```
      />
      {errors.name && <p className="error">{errors.name.message}</p>}

      <input
        placeholder="Email"
        {...register("email", {
          required: "Email is required",
          pattern: { value: /^\S+@\S+\.\S+$/, message: "Invalid email" },
        })}
      />
      {errors.email && <p className="error">{errors.email.message}</p>}

      <input
        type="password"
        placeholder="Password"
        {...register("password", { required: "Password is required", minLength: { value: 6, messa
      />
      {errors.password && <p className="error">{errors.password.message}</p>}

      <input
        type="password"
        placeholder="Confirm password"
        {...register("confirm", {
          validate: (value) => value === password || "Passwords do not match",
        })}
      />
      {errors.confirm && <p className="error">{errors.confirm.message}</p>}

      <button disabled={isSubmitting}>{isSubmitting ? "Saving..." : "Sign up"}</button>
    </form>
  );
}
```

💻 Schema Validation with Zod (recommended for bigger forms)

```
cd C:\Users\YourName\Projects\my-app
npm install zod @hookform/resolvers
```

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

// One schema = all validation rules (also reusable on the backend)
const schema = z.object({
  email: z.string().email("Invalid email"),
  password: z.string().min(6, "Min 6 characters"),
  age: z.coerce.number().min(18, "Must be 18+"),     // coerce: input text → number
});
```

```
function LoginForm() {
  const { register, handleSubmit, formState: { errors } } = useForm({
    resolver: zodResolver(schema),
  });

  return (
    <form onSubmit={handleSubmit((data) => console.log(data))}>
      <input {...register("email")} />
      <p>{errors.email?.message}</p>
      <input type="password" {...register("password")} />
      <p>{errors.password?.message}</p>
      <input type="number" {...register("age")} />
      <p>{errors.age?.message}</p>
      <button>Login</button>
    </form>
  );
}
```

🧠 Manual Controlled Form vs React Hook Form

	Manual (Part 1)	React Hook Form
Code per field	State + handler + value	One `register()`
Re-renders	Every keystroke	✅ Minimal
Validation	Hand-written	Built-in rules or Zod schema
Errors	Manual state	`formState.errors`
Best for	Tiny forms	Medium/large forms

✅ Key Points

- RHF reduces **boilerplate** and **re-renders** by using **uncontrolled inputs** under the hood.
- `register("fieldName", rules)` connects an input; spread it: `{...register("email")}`.
- `handleSubmit(onSubmit)` validates first, then calls `onSubmit(data)` only if valid.
- `formState.errors` holds messages; `isSubmitting` helps disable the button during the request.
- Built-in rules: `required`, `minLength`, `maxLength`, `min`, `max`, `pattern`, `validate`.
- `validate` is for custom checks (e.g., password confirmation); return `true` or an error message.
- `watch("field")` reads a live value; `reset()` clears or restores the form; `setValue` sets one field.

- `defaultValues` in `useForm` set the initial values.
- **Zod +** `zodResolver` moves all rules into **one schema**, easier to maintain and reusable on the backend.
- Use `z.coerce.number()` because HTML inputs always give **strings**.
- For custom components (e.g., UI library selects), use `Controller` to connect them.
- Add `noValidate` on `<form>` to turn off the browser's own validation bubbles.

> 💡 **Memory tip:** RHF is a clipboard collector: it quietly gathers every field's value, and only checks them when you press submit.

> 🎤 **Say it in one line:** "React Hook Form handles form state and validation with minimal code and re-renders using register and handleSubmit."

3️⃣9️⃣ Authentication Flow

> **How users log in and stay logged in: JWT, token storage, and protected API calls.**

📖 Explanation

Authentication = "who are you?" The server checks credentials and gives the client a **JWT (JSON Web Token)**, a signed string proving identity. The client sends it with every protected request, and the server verifies it.

🔍 What's Inside a JWT

```
xxxxx.yyyyy.zzzzz
  |     |     └── Signature (proves it wasn't tampered with)
  |     └──────── Payload  { userId, role, exp }  ← readable by anyone (NOT encrypted!)
  └────────────── Header   { alg: "HS256", typ: "JWT" }
```

🔄 The Full Flow

```
1. User submits email + password ──────────► POST /api/auth/login
2. Server checks (bcrypt compare) ─ OK ─────► creates JWT, sends it back
3. Client stores the token (state + storage)
4. Client calls protected APIs with header ──► Authorization: Bearer <token>
5. Server middleware verifies the token ─ OK ─► returns data  (else 401 Unauthorized)
6. On app reload: client reads the saved token ► asks /auth/me ► restores the user
7. Logout: client deletes the token / server clears the cookie
```

💻 Backend (Express) — Register, Login, Protect

```
// server/routes/auth.js
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const router = express.Router();

router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  if (await User.findOne({ email })) return res.status(400).json({ message: "Email already used"

  const hashed = await bcrypt.hash(password, 10);          // NEVER store plain passwords
  const user = await User.create({ name, email, password: hashed });
  res.status(201).json({ id: user._id, name, email });
});

router.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.password)))
    return res.status(401).json({ message: "Invalid credentials" });   // same message for both c

  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "1h" }
  );
  res.json({ token, user: { id: user._id, name: user.name, role: user.role } });
});

module.exports = router;
```

```
// server/middleware/auth.js — protects routes
const jwt = require("jsonwebtoken");

module.exports = function auth(req, res, next) {
  const header = req.headers.authorization;                // "Bearer eyJhbGci..."
  if (!header?.startsWith("Bearer ")) return res.status(401).json({ message: "No token" });

  try {
    req.user = jwt.verify(header.split(" ")[1], process.env.JWT_SECRET);   // { id, role, iat, ex
    next();
  } catch {
    res.status(401).json({ message: "Invalid or expired token" });
  }
};
```

```
// Usage: router.get("/me", auth, (req, res) => { ... });
```

💻 Frontend — Axios Setup (token + auto-logout on 401)

```
// src/api.js
import axios from "axios";

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";                    // expired/invalid → force login
    }
    return Promise.reject(err);
  }
);

export default api;
```

> The `AuthProvider` (login, logout, restore session with `loading`) and `ProtectedRoute` were built in **Part 3 (Topics 25 and 27)**. They plug straight into this setup.

🔐 Where to Store the Token?

Option	How it works	Security
`localStorage`	Easy: read/write with JS	⚠️ Any **XSS** attack can steal it
`httpOnly` **cookie**	Server sets it; JS **cannot read** it	✅ Safer against XSS; needs **CSRF** protection (`SameSite`)
Memory (state) + refresh cookie	Short-lived access token in memory, refresh token in an httpOnly cookie	✅ Best practice

```
// httpOnly cookie approach (server side): res.cookie("token", token, { httpOnly: true, secure: t
// Client side: api = axios.create({ baseURL, withCredentials: true })   // sends cookies automat
```

✅ Key Points

- **Authentication** = who you are; **Authorization** = what you're allowed to do (roles/permissions).
- A **JWT** has 3 parts: **header.payload.signature**; the payload is **encoded, NOT encrypted**, so never put secrets in it.
- The server **signs** the JWT with a **secret** (`JWT_SECRET`); only the server can create a valid one.
- Login flow: **credentials → server verifies → token returned → client stores → client sends it on each request**.
- Send the token in the header: `Authorization: Bearer <token>`.
- **Hash passwords** with **bcrypt**; never store or log plain passwords.
- Use a **generic message** ("Invalid credentials") so attackers can't tell whether the email exists.
- Set a token **expiry** (`expiresIn`); handle `401` globally in an Axios **interceptor**.
- On page reload, **restore the session** by reading the saved token and calling a `/me` endpoint.
- `localStorage` is simple but vulnerable to **XSS**; **httpOnly cookies** are safer (use `withCredentials: true`).
- Best practice: **short-lived access token + refresh token** (httpOnly cookie).
- The backend must **verify the token on every protected route** (middleware); frontend guards are only for UX.
- Always use **HTTPS** in production, and keep `JWT_SECRET` in **environment variables**, never in code or Git.
- Logout = **delete the token** (and clear the cookie on the server), then clear user state.

> 💡 **Memory tip:** A JWT is a signed concert wristband: show it at every gate (request), staff check the signature, and it stops working when it expires.
>
> 🎤 **Say it in one line:** "The server issues a signed JWT at login, the client sends it with each request, and the server verifies it before returning protected data."

4️⃣0️⃣ Connecting React to a Backend (MERN)

> **React (frontend) ⇄ Express/Node (API) ⇄ MongoDB (database): how the full request cycle works.**

📖 Explanation

MERN = **M**ongoDB + **E**xpress + **R**eact + **N**ode. React runs in the **browser**, Express runs on a **server** and exposes an **API**, and Mongoose talks to **MongoDB**. They're separate programs that communicate over **HTTP**.

🔄 The Full Request Cycle

```
React (browser, :5173)              Express (server, :5000)          MongoDB
──────────────────────              ──────────────────────           ────────

1. User clicks "Add Todo"
2. axios.post("/api/todos") ──────► 3. CORS check ✓
                                    4. Route → controller
                                    5. Mongoose: Todo.create() ──────► 6. Saves document
                                    8. res.json(todo) ◄──────────────── 7. Returns saved d
9. React updates state / cache
10. UI re-renders with the new todo
```

📁 Project Structure

```
mern-app/
├── client/                 ← React (Vite)
│   ├── .env                ← VITE_API_URL=http://localhost:5000/api
│   └── src/ (api.js, pages/, components/, context/)
└── server/                 ← Express API
    ├── .env                ← PORT, MONGO_URI, JWT_SECRET
    ├── server.js
    ├── models/Todo.js
    ├── routes/todos.js
    └── middleware/auth.js
```

💻 Backend — Server, CORS, DB

```
cd C:\Users\YourName\Projects\mern-app\server
npm install express mongoose cors dotenv
```

```
// server/server.js
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
```

```
const app = express();

app.use(cors({
  origin: process.env.CLIENT_URL || "http://localhost:5173",   // ONLY allow your frontend
  credentials: true,                                           // needed if using cookies
}));
app.use(express.json());                                       // parse JSON request bodies

app.use("/api/todos", require("./routes/todos"));

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT || 5000, () => console.log("Server running")))
  .catch((err) => console.error("DB connection failed:", err));
```

```
// server/models/Todo.js
const mongoose = require("mongoose");

const todoSchema = new mongoose.Schema(
  { text: { type: String, required: true, trim: true }, done: { type: Boolean, default: false } },
  { timestamps: true }
);
module.exports = mongoose.model("Todo", todoSchema);
```

```
// server/routes/todos.js
const router = require("express").Router();
const Todo = require("../models/Todo");

router.get("/", async (req, res) => res.json(await Todo.find().sort({ createdAt: -1 })));

router.post("/", async (req, res) => {
  if (!req.body.text) return res.status(400).json({ message: "Text required" });
  res.status(201).json(await Todo.create({ text: req.body.text }));
});

router.put("/:id", async (req, res) => {
  const todo = await Todo.findByIdAndUpdate(req.params.id, req.body, { new: true });
  if (!todo) return res.status(404).json({ message: "Not found" });
  res.json(todo);
});

router.delete("/:id", async (req, res) => {
  await Todo.findByIdAndDelete(req.params.id);
  res.status(204).end();
});

module.exports = router;
```

💻 Frontend — Env Variable, Axios, Component

```
# client/.env   (Vite only exposes variables that START with VITE_)
VITE_API_URL=http://localhost:5000/api
```

```
// client/src/api.js
import axios from "axios";
export default axios.create({ baseURL: import.meta.env.VITE_API_URL });
```

```
// client/src/Todos.jsx — using TanStack Query (Topic 37)
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { useState } from "react";
import api from "./api";

function Todos() {
  const qc = useQueryClient();
  const [text, setText] = useState("");

  const { data: todos = [], isPending, isError } = useQuery({
    queryKey: ["todos"],
    queryFn: async () => (await api.get("/todos")).data,
  });

  const addTodo = useMutation({
    mutationFn: (text) => api.post("/todos", { text }),
    onSuccess: () => { qc.invalidateQueries({ queryKey: ["todos"] }); setText(""); },
  });

  const toggle = useMutation({
    mutationFn: (t) => api.put(`/todos/${t._id}`, { done: !t.done }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ["todos"] }),
  });

  const remove = useMutation({
    mutationFn: (id) => api.delete(`/todos/${id}`),
    onSuccess: () => qc.invalidateQueries({ queryKey: ["todos"] }),
  });

  if (isPending) return <p>Loading...</p>;
  if (isError) return <p>Could not load todos.</p>;

  return (
    <>
      <form onSubmit={(e) => { e.preventDefault(); addTodo.mutate(text); }}>
        <input value={text} onChange={(e) => setText(e.target.value)} />
        <button disabled={!text.trim() || addTodo.isPending}>Add</button>
      </form>
      <ul>
        {todos.map((t) => (
```

```
          <li key={t._id}>
            <span onClick={() => toggle.mutate(t)} style={{ textDecoration: t.done ? "line-throug
              {t.text}
            </span>
            <button onClick={() => remove.mutate(t._id)}>❌</button>
          </li>
        ))}
      </ul>
    </>
  );
}
```

💻 Alternative to CORS in Development: Vite Proxy

```
// client/vite.config.js — forwards /api calls to the server (no CORS issue in dev)
export default defineConfig({
  plugins: [react()],
  server: { proxy: { "/api": "http://localhost:5000" } },
});
// Then in React just call:  axios.get("/api/todos")
```

🧠 Common Errors & Fixes

Error	Cause	Fix
CORS error in console	Server doesn't allow your frontend's origin	`app.use(cors({ origin: CLIENT_URL }))` or Vite proxy
`404` on API call	Wrong URL / missing `/api` prefix	Check `baseURL` and route paths
`req.body` is `undefined`	Missing body parser	Add `app.use(express.json())`
`import.meta.env.X` is `undefined`	Variable doesn't start with `VITE_` or server not restarted	Rename to `VITE_X`, restart `npm run dev`
`401 Unauthorized`	Missing/expired token	Attach `Authorization` header via interceptor
Works locally, fails in production	Hard-coded `localhost` URLs	Use env variables for both client and server

✅ Key Points

- **MERN** = MongoDB (database) + Express (API) + React (UI) + Node (runtime).
- React (browser) and Express (server) are **separate apps** talking over **HTTP/JSON**.
- Typical dev ports: React **5173**, Express **5000**: different origins.
- **CORS** is a **browser** security rule: the server must **explicitly allow** your frontend's origin (`cors` package).
- Use `express.json()` to read JSON request bodies.
- **Mongoose models** define the shape of MongoDB documents; routes call model methods (`find`, `create`, ...).
- Use **REST conventions**: `GET` read, `POST` create, `PUT/PATCH` update, `DELETE` remove.
- Return proper **status codes**: `200` OK, `201` Created, `400` Bad request, `401` Unauthorized, `404` Not found, `500` Server error.
- On the client, create **one Axios instance** with `baseURL` from `import.meta.env.VITE_API_URL`.
- Vite only exposes env variables starting with `VITE_`; **restart** the dev server after changing `.env`.
- Keep **secrets** (`MONGO_URI`, `JWT_SECRET`) on the **server** in `.env`; never in React code (anything in the frontend is public).
- Add `.env` **to** `.gitignore`.
- In dev, a **Vite proxy** can replace CORS setup; in production, set the CORS `origin` to your deployed frontend URL.
- For data fetching in the UI, pair Axios with **TanStack Query** (cache, loading states, invalidation).
- Debug order when something fails: **browser Network tab → server logs → DB**.

> 💡 **Memory tip:** React is the customer, Express is the waiter, MongoDB is the kitchen. The customer never enters the kitchen, only talks to the waiter (API).
>
> 🎤 **Say it in one line:** "React sends HTTP requests to an Express API, which uses Mongoose to read and write MongoDB, and returns JSON."

📝 Part 5 Cheat Sheet

Tool	Purpose	Core API
Redux Toolkit	Large global client state	`createSlice`, `configureStore`, `useSelector`, `useDispatch`, `createAsyncThunk`
Zustand	Simple global client state	`create((set, get) => ({...}))`, selector hook

Tool	Purpose	Core API
TanStack Query	Server state (fetch + cache)	`useQuery({queryKey, queryFn})`, `useMutation`, `invalidateQueries`
React Hook Form	Forms + validation	`useForm`, `register`, `handleSubmit`, `formState.errors`, `zodResolver`
JWT auth	Identity between client and server	`Authorization: Bearer <token>`, verify middleware, interceptors
MERN link	Frontend ⇄ API ⇄ DB	Axios `baseURL`, `VITE_API_URL`, `cors`, `express.json()`

🧭 Which State Tool Do I Use?

Situation	Choose
A toggle inside one component	`useState`
Theme / current user across the app	Context
Cart, filters, many components updating shared data	Zustand (small/medium) or Redux Toolkit (large)
Data from an API (list, details, caching)	TanStack Query
A big form with validation	React Hook Form (+ Zod)

🔥 Top 10 Quick-Fire Questions

1. **Why Redux Toolkit instead of plain Redux?** → Less boilerplate: `createSlice`, Immer, built-in thunk and DevTools.
2. **Why can you "mutate" state in `createSlice`?** → RTK uses Immer, which converts it into immutable updates.
3. **What does `createAsyncThunk` generate?** → `pending`, `fulfilled` and `rejected` actions for the async call.
4. **Zustand vs Redux?** → Zustand is smaller and simpler, with no provider and no reducers; Redux is more structured for big apps.
5. **What is server state?** → Data that lives on the server and can go stale; TanStack Query caches and syncs it.

6. `staleTime` **vs** `gcTime`**?** → staleTime = how long data is fresh; gcTime = how long unused data stays in the cache.
7. **Why use React Hook Form?** → Less code and fewer re-renders, with easy validation.
8. **Is a JWT encrypted?** → No. It's signed, not encrypted; never put secrets in the payload.
9. **Where should the token be stored?** → An httpOnly cookie is safest against XSS; localStorage is simple but vulnerable.
10. **What causes a CORS error and how do you fix it?** → The browser blocks a response because the server doesn't allow the frontend's origin; configure `cors` on the server (or use a dev proxy).

✅ **Part 5 complete.** Next: **Part 6 — Modern React & Extras** (Topics 41–46), the final part.

🚀 React Revision Course — Part 4: Performance & Quality

> **Topics 28–34** · Target: ~10 pages · Simple English · Revise from zero
>
> **How to use this file:** Read the *explanation*, run the *code*, then cover the page and say the *key points* out loud. If you can explain 7 out of 10 points, you've absorbed the topic.

📑 Contents

#	Topic	#	Topic
28	React.memo	32	Refs & forwardRef
29	Code Splitting	33	HOCs & Render Props
30	Error Boundaries	34	Performance Optimization
31	Portals		**Cheat Sheet** (end)

2️⃣8️⃣ React.memo

> **Skip re-rendering a component when its props haven't changed.**

📖 Explanation

By default, when a parent re-renders, **all its children re-render too**, even if their props are identical. Wrapping a component in `React.memo` makes React compare the new props with the old ones (**shallow comparison**) and **skip the render if nothing changed**.

🔄 Without vs With memo

```
Parent state changes → Parent re-renders
   ├── Child (normal)        → re-renders ALWAYS
   └── Child (React.memo)    → re-renders ONLY if its props changed
```

💻 Code

```
import { memo, useCallback, useState } from "react";

// Wrap the child in memo
const ExpensiveList = memo(function ExpensiveList({ items, onSelect }) {
  console.log("ExpensiveList rendered");
  return (
    <ul>
      {items.map((i) => (
        <li key={i.id} onClick={() => onSelect(i.id)}>{i.name}</li>
      ))}
    </ul>
  );
});

function Parent() {
  const [count, setCount] = useState(0);
  const [items] = useState([{ id: 1, name: "A" }, { id: 2, name: "B" }]);

  // ✅ Stable function reference, so memo can do its job
  const handleSelect = useCallback((id) => console.log("selected", id), []);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
      {/* Clicking the button re-renders Parent, but ExpensiveList is SKIPPED */}
      <ExpensiveList items={items} onSelect={handleSelect} />
    </>
  );
}
```

💻 Code — Custom Comparison (rare)

```
const Row = memo(
  function Row({ user }) { return <p>{user.name}</p>; },
  (prevProps, nextProps) => prevProps.user.id === nextProps.user.id   // true = skip render
);
```

⚠️ When memo Does NOT Help

```
// ❌ New object / array / function on every render → props "changed" every time
<ExpensiveList items={items.filter(x => x.ok)} onSelect={(id) => doIt(id)} />

// ✅ Stabilize them
const visible = useMemo(() => items.filter((x) => x.ok), [items]);
const onSelect = useCallback((id) => doIt(id), []);
<ExpensiveList items={visible} onSelect={onSelect} />
```

```
// ❌ Passing JSX as children → new element object every render
<MemoCard><p>Hi</p></MemoCard>
```

✅ Key Points

- `React.memo(Component)` makes a component **skip re-rendering if its props are shallow-equal** to last time.
- By default, a **parent re-render re-renders all children**; memo stops that for children.
- **Shallow comparison** checks references: `{}` is **not equal** to `{}` (a new object each render).
- Memo only helps if props are **stable**: use `useMemo` for objects/arrays and `useCallback` for functions.
- Memo does **not** block re-renders caused by the component's **own state** or by **Context** it consumes.
- You can pass a **custom comparison function** as the 2nd argument (returns `true` to skip).
- **Don't wrap everything**: comparing props has a cost; use it for **heavy components** or **big lists** that re-render often.
- It is purely a **performance optimization**, not a guarantee (React may still re-render).
- Measure first with the **React DevTools Profiler**.
- Name the function (`memo(function Name() {...})`) so DevTools shows a readable component name.

> 💡 **Memory tip:** memo is a bouncer: "Same props as last time? Not coming in."
>
> 🎤 **Say it in one line:** "React.memo skips re-rendering a component when its props haven't changed."

2️⃣9️⃣ Code Splitting

> **Load code only when it's needed, so the first page loads faster.**

📖 Explanation

Normally, the bundler combines your whole app into **one big JavaScript file**, so users download everything upfront. **Code splitting** breaks it into **chunks** loaded **on demand**. In React you do it with `React.lazy` + `Suspense`.

🔄 Flow

```
User opens "/"          → downloads main bundle only
User clicks "/dashboard" → downloads the Dashboard chunk then → renders it
                           (shows <Suspense fallback> while waiting)
```

💻 Code — Route-Based Splitting (most common)

```
import { lazy, Suspense } from "react";
import { Routes, Route } from "react-router";

// The import() runs only when the component is first rendered
const Home = lazy(() => import("./pages/Home"));
const Dashboard = lazy(() => import("./pages/Dashboard"));
const Settings = lazy(() => import("./pages/Settings"));

function App() {
  return (
    <Suspense fallback={<p>Loading page...</p>}>   {/* shown while a chunk downloads */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </Suspense>
  );
}
```

💻 Code — Component-Level & Named Exports

```
const HeavyChart = lazy(() => import("./HeavyChart"));

function Report() {
  const [show, setShow] = useState(false);
  return (
    <>
      <button onClick={() => setShow(true)}>Show chart</button>
      {show && (
        <Suspense fallback={<p>Loading chart...</p>}>
          <HeavyChart />
        </Suspense>
      )}
    </>
  );
}

// lazy() expects a DEFAULT export. For a named export, map it:
const Chart = lazy(() =>
```

```
  import("./charts").then((module) => ({ default: module.Chart }))
);
```

✅ Key Points

- Code splitting = **dividing the bundle into chunks** loaded when needed.
- Benefit: **smaller initial download → faster first load**.
- `React.lazy(() => import("./X"))` loads a component on demand.
- `<Suspense fallback={...}>` shows a placeholder while the lazy component loads.
- A `Suspense` can wrap **many lazy components**; one fallback covers them all.
- `lazy` expects a **default export**; for named exports, return `{ default: module.Name }`.
- **Route-based splitting** is the best starting point (each page = a chunk).
- Also split **heavy, rarely-used components**: charts, editors, modals, admin panels.
- `lazy` components must be defined **outside** other components (top level of the file).
- Combine with an **Error Boundary** to handle chunk-load failures (e.g., offline).
- Bundlers like **Vite/Webpack** create the chunks automatically from `import()`.

> 💡 **Memory tip:** Don't carry the whole library to class; borrow each book only when you need it.
>
> 🎤 **Say it in one line:** "Code splitting with lazy and Suspense loads components on demand to reduce the initial bundle size."

3️⃣0️⃣ Error Boundaries

> **Catch JavaScript errors in the component tree and show a fallback UI instead of a blank screen.**

📖 Explanation

If a component throws an error while **rendering**, React **unmounts the whole app** (white screen). An **Error Boundary** is a component that **catches** such errors in its children and shows a **fallback UI**. Error boundaries must be **class components** (there is no hook version).

🔄 Behavior

```
Without boundary:  Error in <Widget /> → entire app crashes → blank page
With boundary:     Error in <Widget /> → boundary shows fallback → rest of app works
```

💻 Code — Writing One

```jsx
import { Component } from "react";

class ErrorBoundary extends Component {
  state = { hasError: false, error: null };

  // 1️⃣ Update state so the next render shows the fallback
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // 2️⃣ Log the error (send to a monitoring service)
  componentDidCatch(error, info) {
    console.error("Caught:", error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return (
        <div>
          <h2>Something went wrong 😢</h2>
          <button onClick={() => this.setState({ hasError: false })}>Try again</button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

💻 Code — Using It

```jsx
<ErrorBoundary>
  <App />                {/* top-level: last line of defense */}
</ErrorBoundary>

<ErrorBoundary>
  <Sidebar />            {/* granular: a broken widget doesn't kill the page */}
</ErrorBoundary>
<ErrorBoundary>
  <Feed />
</ErrorBoundary>
```

💻 Code — The Easy Way (`react-error-boundary` library)

```
cd C:\Users\YourName\Projects\my-app
npm install react-error-boundary
```

```jsx
import { ErrorBoundary } from "react-error-boundary";

function Fallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert">
      <p>Error: {error.message}</p>
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  );
}

<ErrorBoundary FallbackComponent={Fallback} onReset={() => { /* reset app state */ }}>
  <Dashboard />
</ErrorBoundary>
```

🚫 What Error Boundaries Do NOT Catch

```
// ❌ Event handlers  → use try/catch inside the handler
// ❌ Async code      → setTimeout, fetch, promises (use try/catch / .catch)
// ❌ Server-side rendering errors
// ❌ Errors thrown in the boundary itself
```

✅ Key Points

- An error during **rendering** with no boundary **crashes the whole app**.
- An **Error Boundary** catches errors in its **child tree** and renders a **fallback UI**.
- It must be a **class component** that defines `getDerivedStateFromError` and/or `componentDidCatch`.
- `getDerivedStateFromError`: update state to show the fallback (runs during render).
- `componentDidCatch`: for **side effects** like logging to an error-tracking service.
- Boundaries catch errors in **rendering, lifecycle methods, and constructors** of children.
- They do **not** catch errors in **event handlers, async code, SSR, or the boundary itself**.
- Use **try/catch** for event handlers and async code.
- Place boundaries **granularly** (per section/widget) plus one at the **top level**.
- Provide a **reset/retry** option for the user.
- The `react-error-boundary` library gives a ready-made, hook-friendly version.
- Also wrap **lazy-loaded** components in a boundary to handle failed chunk loads.

> 💡 **Memory tip:** A boundary is a circuit breaker: one broken appliance trips its own switch, not the whole house.
>
> 🎤 **Say it in one line:** "An error boundary catches render errors in its children and shows a fallback instead of crashing the app."

3️⃣1️⃣ Portals

> **Render a component's output into a different place in the DOM, outside its parent.**

📖 Explanation

Normally a component's HTML appears **inside** its parent's HTML. But **modals, tooltips and dropdowns** can get cut off by a parent's `overflow: hidden` or hidden behind another element's `z-index`. A **portal** renders the children into another DOM node (usually `document.body`) while keeping them **logically** in the React tree.

🔄 Picture

```
React tree:  App → Page → Card → Modal        (Modal is a child of Card)
Real DOM:    <div id="root"> … <div class="card"> … </div> </div>
             <div class="modal-overlay">…</div>   ← Modal's HTML lives HERE (body)
```

💻 Code — Modal

```
import { createPortal } from "react-dom";
import { useEffect } from "react";

function Modal({ isOpen, onClose, children }) {
  // Close on Escape key
  useEffect(() => {
    if (!isOpen) return;
    const onKey = (e) => e.key === "Escape" && onClose();
    document.addEventListener("keydown", onKey);
    return () => document.removeEventListener("keydown", onKey);
  }, [isOpen, onClose]);

  if (!isOpen) return null;

  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="modal" onClick={(e) => e.stopPropagation()}>   {/* don't close when clickir
        {children}
        <button onClick={onClose}>Close</button>
      </div>
    </div>,
    document.body            // ← where to render
  );
}

// Usage
```

```jsx
function Page() {
  const [open, setOpen] = useState(false);
  return (
    <div style={{ overflow: "hidden" }}>   {/* would clip a normal modal */}
      <button onClick={() => setOpen(true)}>Open</button>
      <Modal isOpen={open} onClose={() => setOpen(false)}>
        <h2>Hello from a portal!</h2>
      </Modal>
    </div>
  );
}
```

```css
.overlay { position: fixed; inset: 0; background: rgba(0,0,0,.5);
           display: grid; place-items: center; z-index: 1000; }
.modal   { background: white; padding: 24px; border-radius: 8px; }
```

✅ Key Points

- `createPortal(children, domNode)` renders children into a **different DOM node**.
- Import it from `react-dom` (not `react`).
- Used for **modals, tooltips, dropdowns, toasts**, anything that must escape its parent's layout.
- Solves problems with `overflow: hidden`, `z-index`, and **stacking contexts**.
- Even though the HTML is elsewhere, the portal stays in the **React tree**: it keeps **context** and **state** of its parent.
- **Events bubble through the React tree**, not the DOM tree: a click in a portal bubbles to its React parent.
- Use `e.stopPropagation()` to stop clicks inside the modal from triggering the overlay's close handler.
- Return `null` when closed so nothing is rendered.
- Add **Escape-key closing** and focus handling for **accessibility**.
- The target can be `document.body` or a dedicated `<div id="modal-root">` in `index.html`.

> 💡 **Memory tip:** A portal is a teleport: the component stays in the family tree (React) but appears in a different room (DOM).
>
> 🎤 **Say it in one line:** "A portal renders children into another DOM node while keeping them in the same React tree."

3️⃣2️⃣ Refs & `forwardRef`

> Let a parent get a ref to an element or method *inside* a child component.

📖 Explanation

A `ref` on a **DOM tag** (`<input ref={r} />`) gives you the DOM node. But on a **custom component**, a `ref` doesn't automatically reach the inside. `forwardRef` (React 18) lets a component **receive and pass along** the `ref` to an inner element. In **React 19**, `ref` is simply a **normal prop**, so `forwardRef` is no longer needed.

💻 Code — React 18 (`forwardRef`)

```
import { forwardRef, useRef } from "react";

// Child: second argument `ref` is the forwarded ref
const FancyInput = forwardRef(function FancyInput({ label, ...props }, ref) {
  return (
    <label>
      {label}
      <input ref={ref} {...props} />    {/* attach it to the real <input> */}
    </label>
  );
});

function Form() {
  const inputRef = useRef(null);
  return (
    <>
      <FancyInput ref={inputRef} label="Name" />
      <button onClick={() => inputRef.current.focus()}>Focus the input</button>
    </>
  );
}
```

💻 Code — React 19 (`ref` as a prop)

```
function FancyInput({ label, ref, ...props }) {      // ref arrives like any prop
  return (
    <label>
      {label}
      <input ref={ref} {...props} />
    </label>
  );
}
```

💻 Code — Callback Refs & Refs in Lists

```
// Callback ref: a function React calls with the DOM node
<input ref={(node) => node && node.focus()} />

// Many elements: store them in a Map via ref callbacks
function List({ items }) {
  const nodes = useRef(new Map());

  return items.map((item) => (
    <li
      key={item.id}
      ref={(node) => {
        if (node) nodes.current.set(item.id, node);
        else nodes.current.delete(item.id);        // cleanup when removed
      }}
    >
      {item.name}
    </li>
  ));
}
// Later: nodes.current.get(5).scrollIntoView();
```

💻 Code — Expose Only Chosen Methods

```
// (Recap of useImperativeHandle from Part 2)
useImperativeHandle(ref, () => ({
  focus: () => inputRef.current.focus(),     // parent can only call focus(), not touch the DOM
}));
```

✅ Key Points

- A ref on a **DOM element** (`<input ref={r} />`) gives access to the **real DOM node**.
- A ref on a **custom component** does **not** reach inside unless the component **forwards** it.
- **React 18:** wrap the component in `forwardRef((props, ref) => ...)`.
- **React 19:** `ref` is a **regular prop**; `forwardRef` is no longer required (it still works in older code).
- Forward the ref to the **specific inner element** the parent should control.
- **Callback refs** (`ref={(node) => ...}`) let you run code when the node is attached or removed.
- To track **many elements** (lists), store nodes in a **Map** or an object using callback refs.
- Use `useImperativeHandle` to expose **limited methods** instead of the whole DOM node.
- Typical uses: **focus**, **scroll into view**, **measure size**, **play/pause media**, **text selection**.
- Refs are an **escape hatch**; prefer props and state for anything that affects the UI.
- Don't read/write `ref.current` **during rendering**; do it in effects or event handlers.

> 💡 **Memory tip:** forwardRef is a "pass-through" cable: the parent plugs into the child's outside, and the signal reaches the inner element.
>
> 🎤 **Say it in one line:** "forwardRef (or ref as a prop in React 19) lets a parent access a DOM node inside a child component."

3️⃣3️⃣ HOCs & Render Props

> **Two older patterns for sharing logic between components. Hooks have mostly replaced them.**

📖 Explanation

Before hooks, these were the two main ways to **reuse logic**. You'll still meet them in **older codebases and libraries** (like Redux's `connect`), and in interviews.

🔹 Higher-Order Component (HOC): a function that takes a component and returns a new one

```
// HOC: adds auth checking to ANY component
function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    const { user } = useAuth();
    if (!user) return <Navigate to="/login" />;
    return <WrappedComponent {...props} user={user} />;   // pass props through
  };
}

// Usage
function Profile({ user }) { return <h1>{user.name}</h1>; }
const ProtectedProfile = withAuth(Profile);       // naming convention: withSomething
```

🔹 Render Props: a component receives a *function* that returns JSX

```
// Component handles the logic, caller decides the UI
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const move = (e) => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener("mousemove", move);
    return () => window.removeEventListener("mousemove", move);
  }, []);
```

```
  return render(pos);                          // call the function with data
}

// Usage
<MouseTracker render={({ x, y }) => <p>Mouse at {x}, {y}</p>} />
```

🔹 The Modern Way: Custom Hook (same logic, no wrappers)

```
function useMouse() {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  useEffect(() => {
    const move = (e) => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener("mousemove", move);
    return () => window.removeEventListener("mousemove", move);
  }, []);
  return pos;
}

const { x, y } = useMouse();    // one line, no nesting
```

🧠 Comparison

	HOC	Render Props	Custom Hook
Form	Function wrapping a component	Component taking a function prop	Plain function
Nesting ("wrapper hell")	❌ Yes	❌ Yes	✅ None
Readability	Medium	Lower	✅ Best
Status today	Legacy / library use	Legacy / niche	✅ Recommended

✅ Key Points

- **HOC** = a **function** that takes a component and **returns an enhanced component**: `withX(Component)`.
- HOC naming convention: `withSomething` (`withAuth`, `withRouter`).
- A HOC must **forward props** (`{...props}`) to the wrapped component.
- **Never create a HOC inside a render**; it creates a new component each time and **resets state**.

- **Render prop** = a prop whose value is a **function that returns JSX** (`render` or `children`).
- Render props let the **parent decide the UI** while the component owns the logic.
- Both patterns reuse logic but cause **deep nesting ("wrapper hell")**.
- **Custom hooks** do the same job more simply, so they are the **modern standard**.
- You still see HOCs in **Redux** `connect`, older React Router, and legacy projects.
- Interview answer: *"Both share logic; hooks replaced them because they avoid wrapper nesting."*

> 💡 **Memory tip:** HOC = a gift-wrapper that adds a bow to any present. Render prop = a stencil where you choose the colors. Hook = just grab the bow.
>
> 🎤 **Say it in one line:** "HOCs and render props share logic through wrapping and function props; custom hooks replaced them."

3️⃣4️⃣ Performance Optimization

> **Find the real cause of slowness, then fix it with the simplest tool first.**

📖 Explanation

React is fast by default. Most slowness comes from **too many unnecessary re-renders**, **huge lists**, or **big bundles**. The golden rule: **measure first, optimize second.**

🔄 Why Does a Component Re-render?

```
A component re-renders when:
  1. Its OWN state changes
  2. Its PARENT re-renders  (even if props are identical!)
  3. A CONTEXT it uses changes
  4. Its props change
```

🧰 The Optimization Ladder (try in this order)

#	Technique	When
1	**Keep state local** (move it down)	State is high up but only one child uses it
2	**Children as props** pattern	A slow subtree re-renders because a parent's state changes

#	Technique	When
3	`React.memo`	Heavy child, stable props
4	`useMemo` / `useCallback`	Stabilize objects/functions for memoized children; heavy calculations
5	**Virtualize long lists**	Hundreds/thousands of rows
6	**Code split** (`lazy`)	Big bundle, slow first load
7	**Debounce / throttle**	Rapid events (search typing, scroll, resize)

💻 Code — 1. Move State Down

```
// ❌ `color` lives in App → typing re-renders <SlowTree /> too
function App() {
  const [color, setColor] = useState("red");
  return (
    <>
      <input value={color} onChange={(e) => setColor(e.target.value)} />
      <p style={{ color }}>Hello</p>
      <SlowTree />
    </>
  );
}

// ✅ Extract the part that uses the state into its own component
function ColorPicker() {
  const [color, setColor] = useState("red");
  return (
    <>
      <input value={color} onChange={(e) => setColor(e.target.value)} />
      <p style={{ color }}>Hello</p>
    </>
  );
}
function App() {
  return (
    <>
      <ColorPicker />      {/* only this re-renders on typing */}
      <SlowTree />
    </>
  );
}
```

💻 Code — 2. Children-as-Props Trick

```
function ColorWrapper({ children }) {
  const [color, setColor] = useState("red");
  return (
    <div style={{ color }}>
      <input value={color} onChange={(e) => setColor(e.target.value)} />
      {children}             {/* children were created by the parent, so they are NOT re-created
    </div>
  );
}

<ColorWrapper>
  <SlowTree />              {/* does not re-render when color changes */}
</ColorWrapper>
```

💻 Code — 5. Virtualize Long Lists (render only what's visible)

```
cd C:\Users\YourName\Projects\my-app
npm install react-window
```

```
import { FixedSizeList } from "react-window";

function BigList({ items }) {           // 10,000 items, but only ~10 DOM nodes
  return (
    <FixedSizeList height={400} width={300} itemCount={items.length} itemSize={35}>
      {({ index, style }) => <div style={style}>{items[index].name}</div>}
    </FixedSizeList>
  );
}
```

💻 Code — 7. Debounce Rapid Input (reuse `useDebounce` from Part 2)

```
const [query, setQuery] = useState("");
const debounced = useDebounce(query, 400);     // update only after the user pauses
useEffect(() => { fetchResults(debounced); }, [debounced]);
```

🔍 How to Measure

1. Install **React Developer Tools** (browser extension).
2. Open the **Profiler** tab → press **Record** → interact → **Stop**.
3. Check **which components rendered** and **how long** each took.
4. Turn on *"Highlight updates when components render"* in DevTools settings.

5. Always test with a **production build** (`npm run build` + `npm run preview`); dev mode is slower.

✅ Key Points

- **Measure first** (Profiler); don't optimize by guessing.
- A component re-renders when **its state, props, a used context changes, or its parent re-renders**.
- **Re-rendering isn't always bad**; React is fast. Fix only **noticeable** slowness.
- **Move state down** to the smallest component that needs it.
- **Children-as-props** keeps slow subtrees from re-rendering.
- Use `React.memo` + stable props (`useMemo` / `useCallback`) for heavy components.
- Use **stable, unique** `key` **s** in lists, never random or index keys for dynamic lists.
- **Virtualize** huge lists (`react-window`, `react-virtualized`) so only visible rows render.
- **Code split** with `lazy` / `Suspense` to shrink the first download.
- **Debounce/throttle** frequent events (typing, scrolling, resizing).
- Avoid **creating new objects/arrays/functions in props** of memoized children.
- Avoid **expensive work during render**; memoize it or move it elsewhere.
- Split big **contexts**; frequently changing data in one context re-renders every consumer.
- Optimize **images** (size, lazy loading, modern formats) as they often cost more than JS.
- The **React Compiler** can auto-memoize code, reducing manual `memo` / `useMemo` / `useCallback`.

> 💡 **Memory tip:** Don't buy a faster car before checking if you're stuck in traffic. Profile first, then fix the actual bottleneck.
>
> 🎤 **Say it in one line:** "I profile to find unnecessary re-renders, then fix them with local state, memoization, virtualization and code splitting."

📝 Part 4 Cheat Sheet

Topic	Remember this
React.memo	Skips re-render if props are shallow-equal; needs stable props (`useMemo` / `useCallback`)
Code splitting	`lazy(() => import("./X"))` + `<Suspense fallback>`; split by route first

Topic	Remember this
Error boundary	Class with `getDerivedStateFromError` + `componentDidCatch`; not for events/async
Portals	`createPortal(jsx, document.body)`; for modals/tooltips; bubbles via React tree
forwardRef	React 18: `forwardRef((props, ref) => ...)`; React 19: `ref` is a normal prop
HOC	`withX(Component)` returns a new component; legacy
Render props	Prop that's a function returning JSX; legacy
Hooks vs HOC/render props	Hooks share logic with no wrapper nesting → modern choice
Performance	Profile → local state → memo → virtualize → code split → debounce

🔁 Re-render Triggers

Trigger	Re-renders?
Own state changes	✅
Parent re-renders	✅ (unless wrapped in `memo` with same props)
Used context value changes	✅ (even with `memo`)
`useRef` value changes	❌

🔥 Top 8 Quick-Fire Questions

1. **Why does a child re-render when its parent does?** → React re-renders all children by default; `React.memo` can skip it if props are unchanged.
2. **Why doesn't `React.memo` work with inline functions?** → A new function is created every render, so props are never shallow-equal.
3. **What does `Suspense` do?** → Shows a fallback UI while a lazy component (or data) is loading.

4. **Why are error boundaries class components?** → Only classes support `getDerivedStateFromError` / `componentDidCatch`.
5. **What can't an error boundary catch?** → Event handler errors, async code, SSR errors, and errors in the boundary itself.
6. **Why use a portal for a modal?** → To escape parent `overflow` / `z-index` problems while staying in the React tree.
7. **Why did hooks replace HOCs?** → They reuse logic without wrapper nesting and are easier to read.
8. **How do you handle a 10,000-item list?** → Virtualize it so only visible rows are rendered.

✅ **Part 4 complete.** Next: **Part 5 — State Management & Ecosystem** (Topics 35–40).

🧭 React Revision Course — Part 3: Routing & Data

> **Topics 24–27** · Target: ~9 pages · Simple English · Revise from zero
>
> **How to use this file:** Read the *explanation*, run the *code*, then cover the page and say the *key points* out loud. If you can explain 7 out of 10 points, you've absorbed the topic.

📑 Contents

#	Topic
24	React Router
25	Protected Routes
26	Data Fetching
27	Context API Patterns
	Cheat Sheet (end)

2️⃣4️⃣ React Router

> **Navigate between "pages" in a Single Page App without reloading the browser.**

📖 Explanation

A React app has **one HTML page**. React Router watches the **URL** and renders the matching component, so `/about` shows `<About />` with no page reload. It also updates the browser's address bar and back/forward buttons.

💻 Install (go to your project folder first)

```
cd C:\Users\YourName\Projects\my-app
```

```
npm install react-router
```

> In v7 the package is `react-router`. The older `react-router-dom` still works and re-exports the same things, so both import styles appear in tutorials.

🔄 How It Works

```
URL changes (/users/5)
        |
        ▼
Router matches the URL against your <Route> list
        |
        ▼
Renders the matching component (<UserPage />) inside the layout
```

💻 Code — Basic Setup

```
// main.jsx — wrap the whole app in BrowserRouter ONCE
import { BrowserRouter } from "react-router";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

```
// App.jsx
import { Routes, Route, Link, NavLink } from "react-router";

function App() {
  return (
    <>
      <nav>
        {/* Link = navigation without page reload (never use plain <a href> for internal links) */}
        <Link to="/">Home</Link>
        {/* NavLink = Link that knows if it's the current page → good for menus */}
        <NavLink to="/about" className={({ isActive }) => (isActive ? "active" : "")}>
          About
        </NavLink>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users/:id" element={<UserPage />} />   {/* :id = URL param */}
        <Route path="*" element={<NotFound />} />            {/* * = 404 catch-all */}
```

```
      </Routes>
    </>
  );
}
```

💻 Code — URL Params, Query Strings & Programmatic Navigation

```
import { useParams, useNavigate, useSearchParams, Navigate } from "react-router";

function UserPage() {
  const { id } = useParams();                    // /users/5 → id = "5" (always a STRING)
  const navigate = useNavigate();
  const [searchParams, setSearchParams] = useSearchParams();

  const tab = searchParams.get("tab") || "info"; // /users/5?tab=posts → "posts"

  return (
    <div>
      <h2>User #{id}</h2>
      <button onClick={() => setSearchParams({ tab: "posts" })}>Posts tab</button>
      <button onClick={() => navigate(-1)}>⬅ Back</button>
      <button onClick={() => navigate("/")}>Go Home</button>
      <button onClick={() => navigate("/login", { replace: true })}>
        Login (replace history)
      </button>
    </div>
  );
}

// Declarative redirect: <Route path="/old" element={<Navigate to="/new" replace />} />
```

💻 Code — Nested Routes & `Outlet` (shared layout)

```
import { Outlet } from "react-router";

// Layout component: renders shared UI + a slot for the child route
function DashboardLayout() {
  return (
    <div className="dashboard">
      <aside>
        <Link to="/dashboard">Overview</Link>
        <Link to="/dashboard/settings">Settings</Link>
      </aside>
      <main>
        <Outlet />          {/* child route renders HERE */}
      </main>
    </div>
  );
```

```
}

<Routes>
  <Route path="/dashboard" element={<DashboardLayout />}>
    <Route index element={<Overview />} />          {/* index = default child (/dashboard) */}
    <Route path="settings" element={<Settings />} /> {/* relative path (/dashboard/settings) */}
  </Route>
</Routes>
```

🧠 Link vs `<a>` vs `useNavigate`

Tool	Use for	Reloads page?
`<Link to>`	Clickable internal link	❌ No
`<NavLink to>`	Menu link that highlights when active	❌ No
`<a href>`	External sites	✅ Yes
`useNavigate()`	Navigate from code (after login, form submit)	❌ No
`<Navigate to>`	Redirect while rendering	❌ No

✅ Key Points

- React Router enables **client-side routing**: URL changes without a full page reload.
- Wrap the app in `<BrowserRouter>` once, usually in `main.jsx`.
- `<Routes>` holds all routes; `<Route path element>` maps a URL to a component.
- Use `<Link>` (not `<a>`) for internal navigation to avoid reloads.
- `<NavLink>` adds an **active** state, ideal for navigation menus.
- **Dynamic segments** like `/users/:id` are read with `useParams()`, and values are always **strings**.
- **Query strings** (`?tab=posts`) are read/written with `useSearchParams()`.
- `useNavigate()` navigates from code; `navigate(-1)` goes back; `{ replace: true }` replaces the history entry.
- `<Navigate to="/x" />` redirects during rendering.
- **Nested routes** share a layout: the parent renders `<Outlet />` where the child appears.
- An `index` route is the default child shown at the parent's path.
- `path="*"` is the catch-all for a **404 page**.
- Child route paths are **relative** (no leading `/`).
- When hosting a SPA, configure the server to **always serve** `index.html` so refreshing `/about` doesn't 404.

> 💡 **Memory tip:** Router is a receptionist: it reads the URL (the address) and sends the visitor to the right room; `Outlet` is the room inside the building.
>
> 🎤 **Say it in one line:** "React Router maps URLs to components and handles navigation without reloading the page."

2️⃣5️⃣ Protected Routes

> **Block pages (like `/dashboard`) from users who aren't logged in, or don't have the right role.**

📖 Explanation

React Router has no built-in "protected route". You build a **guard component** that checks login status: if allowed, it renders the page (via `<Outlet />`); if not, it redirects to `/login`.

🔄 Flow

```
User visits /dashboard
        |
        ▼
 <ProtectedRoute> checks: is user logged in?
     ├── YES → <Outlet /> (show the page)
     └── NO  → <Navigate to="/login" />   (and remember where they wanted to go)
```

💻 Code — Guard Component (uses the `useAuth` hook from Part 2)

```
import { Navigate, Outlet, useLocation } from "react-router";
import { useAuth } from "./AuthContext";

function ProtectedRoute({ allowedRoles }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <p>Checking login...</p>;         // avoid flashing /login on refresh

  if (!user) {
    // Send to login, and remember the page they tried to open
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  // Optional role check
```

```
  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  return <Outlet />;                                    // allowed → render child routes
}
```

💻 Code — Using It

```
<Routes>
  {/* Public routes */}
  <Route path="/" element={<Home />} />
  <Route path="/login" element={<Login />} />

  {/* Protected: any logged-in user */}
  <Route element={<ProtectedRoute />}>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/profile" element={<Profile />} />
  </Route>

  {/* Protected: admin only */}
  <Route element={<ProtectedRoute allowedRoles={["admin"]} />}>
    <Route path="/admin" element={<AdminPanel />} />
  </Route>
</Routes>
```

💻 Code — Redirect Back After Login

```
function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const from = location.state?.from?.pathname || "/dashboard";   // where they wanted to go

  async function handleSubmit(e) {
    e.preventDefault();
    await login(email, password);
    navigate(from, { replace: true });        // replace → Back button won't return to /login
  }
  // ...
}
```

✅ Key Points

- A protected route is a **wrapper component** you write yourself; React Router doesn't provide one.
- It checks auth state (usually from **Context**) and either renders `<Outlet />` or `<Navigate>`.

- Wrapping many routes in **one layout route** (`<Route element={<ProtectedRoute />}>`) avoids repeating the check.
- Handle a `loading` state, or logged-in users get briefly bounced to `/login` on page refresh.
- Use `replace` on redirects so the Back button doesn't trap the user.
- Pass the original location in `state`, then redirect back after login.
- **Role-based access**: compare `user.role` with an `allowedRoles` list.
- ⚠️ **Frontend protection is only for UX.** Users can bypass it, so the **backend must verify** the token on every protected API request.
- The auth user usually comes from a **token** (e.g., JWT) restored from storage on app load (see Part 5).
- The pattern is: **Context (who is the user) + Guard (can they enter) + Router (where to send them)**.

> 💡 **Memory tip:** The protected route is a security guard at the door: no ID → sent to the front desk (login); wrong badge → "unauthorized".
>
> 🎤 **Say it in one line:** "A protected route checks authentication and either renders the page or redirects to login."

2️⃣6️⃣ Data Fetching

> **Get data from an API, and handle the three states every request has: loading, success, error.**

📖 Explanation

React doesn't fetch data itself; you use `fetch` (built into browsers) or a library like **Axios**, and call it inside `useEffect` (or an event handler). Every request goes through **loading → success or error**, and your UI must handle all three.

🔄 The Three States

```
Start ──► LOADING ──► SUCCESS (show data)
                  └──► ERROR   (show message)
```

💻 Code — `fetch` with async/await

```
import { useEffect, useState } from "react";

function Users() {
```

```
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function loadUsers() {
      try {
        const res = await fetch("https://jsonplaceholder.typicode.com/users", {
          signal: controller.signal,
        });
        // ⚠️ fetch does NOT throw on 404/500. You must check res.ok yourself.
        if (!res.ok) throw new Error(`Server error: ${res.status}`);
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadUsers();
    return () => controller.abort();          // cancel if the component unmounts
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;
  if (users.length === 0) return <p>No users found.</p>;

  return <ul>{users.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

💻 Code — Axios (install first)

```
cd C:\Users\YourName\Projects\my-app
npm install axios
```

```
import axios from "axios";

// Create ONE configured instance for the whole app
const api = axios.create({
  baseURL: "http://localhost:5000/api",
  timeout: 10000,
});

// Auto-attach the token to every request
api.interceptors.request.use((config) => {
```

```
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// GET
const { data } = await api.get("/users");

// POST (JSON is automatic)
await api.post("/users", { name: "Ali", email: "ali@mail.com" });

// PUT / DELETE
await api.put(`/users/${id}`, { name: "New Name" });
await api.delete(`/users/${id}`);

// Errors: Axios throws automatically on 4xx / 5xx
try {
  await api.get("/secret");
} catch (err) {
  console.log(err.response?.status);          // e.g. 401
  console.log(err.response?.data?.message);   // server's message
}
```

💻 Code — POST with `fetch`

```
const res = await fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Ali" }),      // must stringify manually
});
const data = await res.json();
```

🧠 `fetch` vs Axios

	`fetch`	**Axios**
Built in	✅ Yes	❌ Install needed
Parse JSON	Manual `res.json()`	Automatic (`data`)
HTTP errors (404/500)	❌ Doesn't throw (check `res.ok`)	✅ Throws automatically
Send JSON body	Manual `JSON.stringify` + header	Automatic
Interceptors / base URL	❌ No	✅ Yes

	`fetch`	**Axios**
Timeout	Manual (`AbortController`)	Built-in option

✅ Key Points

- Fetch in `useEffect` (on load / when a value changes) or in **event handlers** (on click/submit).
- Always track **three states**: `loading`, `error`, and `data`.
- Show a **loading** UI, an **error** message, and an **empty state**.
- `fetch` **does NOT reject on 404/500**; check `res.ok` and throw yourself.
- With `fetch`, call `res.json()` (it returns a Promise) to read the body.
- **Axios** throws on bad status codes, parses JSON automatically, and supports **interceptors** and a `baseURL`.
- Create **one Axios instance** (`axios.create`) and reuse it everywhere.
- Use **interceptors** to attach auth tokens or handle `401` globally.
- The effect callback can't be `async`; define an **async function inside** and call it.
- **Cancel** requests on cleanup with `AbortController` to avoid state updates on unmounted components and **race conditions**.
- Use `try / catch / finally`: `finally` is perfect for `setLoading(false)`.
- Store the base URL in an **environment variable** (`import.meta.env.VITE_API_URL` in Vite).
- Extract repeated logic into a `useFetch` **custom hook** (Part 2), or use **TanStack Query** for caching (Part 5).

> 💡 **Memory tip:** Every request is a restaurant order: *waiting* (loading), *food arrives* (success), or *kitchen's out of it* (error). Your UI must show all three.
>
> 🎤 **Say it in one line:** "I fetch in useEffect, track loading, error and data states, and cancel requests on cleanup."

2️⃣7️⃣ Context API Patterns

> **Practical ways to use Context for auth and theme, without performance traps.**

📖 Explanation

You learned `useContext` in Part 2. Here are the **patterns used in real apps**: a provider component per concern, a custom hook to consume it, memoized values, and `useReducer` for complex state.

💻 Code — Theme Context (persisted)

```
// ThemeContext.jsx
import { createContext, useContext, useEffect, useState } from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState(() => localStorage.getItem("theme") || "light");

  useEffect(() => {
    localStorage.setItem("theme", theme);
    document.documentElement.setAttribute("data-theme", theme);   // CSS can react to it
  }, [theme]);

  const toggleTheme = () => setTheme((t) => (t === "light" ? "dark" : "light"));

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error("useTheme must be used inside <ThemeProvider>");
  return ctx;
}
```

💻 Code — Auth Context (with loading + memoized value)

```
// AuthContext.jsx
import { createContext, useContext, useEffect, useMemo, useState, useCallback } from "react";
import api from "./api";                          // your Axios instance

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);   // true while checking saved token

  // On app start: restore the session
  useEffect(() => {
    async function restore() {
      try {
        const token = localStorage.getItem("token");
        if (token) {
          const { data } = await api.get("/auth/me");
```

```
          setUser(data);
        }
      } catch {
        localStorage.removeItem("token");
      } finally {
        setLoading(false);
      }
    }
    restore();
  }, []);

  const login = useCallback(async (email, password) => {
    const { data } = await api.post("/auth/login", { email, password });
    localStorage.setItem("token", data.token);
    setUser(data.user);
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem("token");
    setUser(null);
  }, []);

  // Memoize the value so consumers only re-render when something REALLY changes
  const value = useMemo(
    () => ({ user, loading, login, logout }),
    [user, loading, login, logout]
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used inside <AuthProvider>");
  return ctx;
}
```

💻 Code — Wire Providers in `main.jsx`

```
createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <ThemeProvider>
      <AuthProvider>
        <App />
      </AuthProvider>
    </ThemeProvider>
  </BrowserRouter>
);
```

💻 Code — Context + `useReducer` (mini Redux)

```
const CartContext = createContext(null);

function cartReducer(items, action) {
  switch (action.type) {
    case "add":    return [...items, action.item];
    case "remove": return items.filter((i) => i.id !== action.id);
    case "clear":  return [];
    default:       return items;
  }
}

export function CartProvider({ children }) {
  const [items, dispatch] = useReducer(cartReducer, []);
  const value = useMemo(() => ({ items, dispatch }), [items]);
  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
}
```

⚠️ Re-render Traps

```
// ❌ New object every render → ALL consumers re-render whenever the provider re-renders
<AuthContext.Provider value={{ user, login }}>

// ✅ Memoize the value
const value = useMemo(() => ({ user, login }), [user, login]);

// ❌ One giant context (theme + user + cart + settings) → any change re-renders everything
// ✅ Split into focused contexts: ThemeContext, AuthContext, CartContext
```

✅ Key Points

- Create **one context per concern** (theme, auth, cart), not one giant context.
- Wrap each context in a **Provider component** that owns its state and logic.
- Expose it through a **custom hook** (`useAuth`, `useTheme`) and **throw an error** if used outside the Provider.
- Provider components accept `children`, so they can wrap the app without extra props.
- **Memoize** the context `value` with `useMemo` (and functions with `useCallback`) to avoid needless re-renders.
- **All consumers re-render** whenever the Provider's `value` changes (even if they use only part of it).
- **Split** contexts (or split state and dispatch into two contexts) to limit re-renders.
- Use a `loading` state in auth so guards don't redirect before the session is restored.
- Persist theme/token in `localStorage`, and initialize state **lazily** from it (`useState(() => ...)`).

- Combine with `useReducer` when state has many actions (cart, notifications).
- Order providers so inner ones can use outer ones (e.g., `AuthProvider` can use the router if inside `BrowserRouter`).
- **Not for everything**: for large or fast-changing global state, use **Redux Toolkit / Zustand** (Part 5).
- Don't store **sensitive secrets** in context or `localStorage`; keep access tokens short-lived and secured (see Part 5).

> 💡 **Memory tip:** One context = one radio channel. Don't broadcast everything on one channel, or every listener gets interrupted for every announcement.
>
> 🎤 **Say it in one line:** "I give each concern its own Provider and custom hook, and memoize the value so only real changes re-render consumers."

📝 Part 3 Cheat Sheet

Topic	Remember this
Router setup	`npm install react-router` → wrap app in `<BrowserRouter>` → `<Routes>` + `<Route path element>`
Navigation	`<Link>` / `<NavLink>` for clicks, `useNavigate()` for code, `<Navigate>` to redirect
URL data	`useParams()` (`/users/:id`, string), `useSearchParams()` (`?tab=x`)
Layouts	Parent route + `<Outlet />`; `index` = default child; `path="*"` = 404
Protected routes	Guard component: `user ? <Outlet /> : <Navigate to="/login" />` + loading state
Backend rule	Frontend guard = UX only; backend must verify token
Fetching	`useEffect` + `loading / error / data` states + cleanup with `AbortController`
fetch gotcha	Doesn't throw on 404/500 → check `res.ok`
Axios	`axios.create({ baseURL })`, interceptors for token, throws on bad status

Topic	Remember this
Context patterns	One context per concern + Provider + custom hook + `useMemo` value

🔥 Top 8 Quick-Fire Questions

1. **Why `<Link>` instead of `<a>`?** → `<Link>` navigates without a full page reload; `<a>` reloads the page.
2. **What is `<Outlet />`?** → The placeholder in a parent route where the matching child route renders.
3. **How do you read `/users/5`?** → Route `path="/users/:id"`, then `const { id } = useParams()` (it's a string).
4. **How do you navigate after login?** → `useNavigate()`, e.g. `navigate("/dashboard", { replace: true })`.
5. **How does a protected route work?** → A guard checks auth; if logged in it renders `<Outlet />`, else `<Navigate to="/login" />`.
6. **Is a frontend guard enough for security?** → No. The server must verify the token on every protected request.
7. **Why doesn't `fetch` catch a 404 in `catch`?** → It only rejects on network failure; check `res.ok` yourself.
8. **Why memoize the Context value?** → A new object each render makes every consumer re-render needlessly.

✅ **Part 3 complete.** Next: **Part 4 — Performance & Quality** (Topics 28–34).

🪝 React Revision Course — Part 2: Hooks

> **Topics 14–23** · Target: ~20 pages · Simple English · Revise from zero
>
> **How to use this file:** Read the *explanation*, run the *code*, then cover the page and say the *key points* out loud. If you can explain 7 out of 10 points, you've absorbed the topic.

📑 Contents

#	Topic	#	Topic
14	What is a Hook + Rules of Hooks	19	useMemo
15	useState	20	useCallback
16	useEffect	21	useReducer
17	useContext	22	Custom Hooks
18	useRef	23	Other Hooks
			Cheat Sheet (end)

1️⃣4️⃣ What is a Hook + Rules of Hooks

> **Special functions (starting with `use`) that let function components use React features like state and lifecycle.**

📖 Explanation

Before 2019 (React 16.8), only **class components** could have state and lifecycle methods. Hooks gave **function components** the same powers, with less code and easier logic reuse.

💻 Code — Class vs Hook

```
// ❌ Old way: class component
class Box extends React.Component {
```

```
  state = { count: 0 };
  render() {
    return <p>{this.state.count}</p>;
  }
}

// ✅ Modern way: function + hook
function Box() {
  const [count] = useState(0);
  return <p>{count}</p>;
}
```

🚦 The Two Golden Rules

```
// RULE 1: Only call Hooks at the TOP LEVEL
function Profile({ show }) {
  const [name, setName] = useState("");        // ✅ top level

  if (show) {
    const [age, setAge] = useState(0);         // ❌ inside a condition
  }
  for (let i = 0; i < 3; i++) {
    useEffect(() => {});                       // ❌ inside a loop
  }
  function helper() {
    const [x, setX] = useState(0);             // ❌ inside a nested function
  }
}

// RULE 2: Only call Hooks from React functions
function MyComponent() { useState(0); }        // ✅ component
function useMyHook() { useState(0); }          // ✅ custom hook
function normalFunction() { useState(0); }     // ❌ plain JS function
```

🧠 Why the Rules Exist

React doesn't know hook names. It tracks hooks **by the order they are called** (1st, 2nd, 3rd…). If the order changes between renders, React matches state to the wrong hook and everything breaks.

```
Render 1: useState(→#1)  useEffect(→#2)  useRef(→#3)
Render 2: useState(→#1)  useEffect(→#2)  useRef(→#3)   ✅ same order
Render 2: useState(→#1)  [skipped]       useRef(→#2)   ❌ mismatch!
```

🧰 Built-in Hooks at a Glance

Group	Hooks
State	`useState`, `useReducer`
Context	`useContext`
Refs	`useRef`, `useImperativeHandle`
Effects	`useEffect`, `useLayoutEffect`
Performance	`useMemo`, `useCallback`, `useTransition`, `useDeferredValue`
Other	`useId`, and your own **custom hooks**

✅ Key Points

- A **Hook** is a function whose name starts with `use`.
- Introduced in **React 16.8 (2019)**.
- Hooks let function components have **state, side effects, context, refs** and more.
- **Rule 1:** call hooks only at the **top level** (never in loops, conditions or nested functions).
- **Rule 2:** call hooks only inside **React function components** or **custom hooks**.
- React identifies hooks by **call order**, so the order must be identical every render.
- Conditions go **inside** a hook (`useEffect(() => { if (x) {...} })`), not around it.
- Hooks solve: reusing stateful logic, avoiding "wrapper hell", and splitting code by purpose, not lifecycle method.
- The **ESLint plugin** `eslint-plugin-react-hooks` warns you when you break the rules.
- You can create your **own hooks** by combining built-in ones (Topic 22).

> 💡 **Memory tip:** Hooks are numbered lockers. React opens locker #1, #2, #3 in order; if you skip one, everything shifts.
>
> 🎤 **Say it in one line:** "Hooks let function components use state and lifecycle, as long as they're called at the top level in the same order."

1️⃣5️⃣ useState

> **Give a component its own memory. Updating it re-renders the component.**

📖 Explanation

`useState(initialValue)` returns an array of two things: the **current value** and a **setter function**. Calling the setter schedules a re-render with the new value.

💻 Code — The Basics

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);   // [value, setter] = useState(initial)

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </>
  );
}
```

💻 Code — Functional Updates (when new value depends on old)

```
function handleClick() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);       // ❌ result: only +1 (all use the SAME old `count`)
}

function handleClickFixed() {
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);   // ✅ result: +3 (each gets the latest value)
}
```

💻 Code — Objects & Arrays (never mutate; make a copy)

```
const [user, setUser] = useState({ name: "Ali", age: 20 });
const [items, setItems] = useState(["a", "b"]);

// Objects
setUser({ ...user, age: 21 });                // ✅ copy + change one field
// user.age = 21;  setUser(user);             // ❌ mutation: React sees the SAME object, no re

// Arrays
setItems([...items, "c"]);                    // ✅ add
setItems(items.filter((i) => i !== "a"));     // ✅ remove
```

```
setItems(items.map((i) => (i === "b" ? "B" : i)));  // ✅ update
// items.push("c");                                 // ❌ mutation
```

💻 Code — Lazy Initialization

```
// ❌ runs expensiveCalc() on EVERY render (result ignored after the first)
const [data, setData] = useState(expensiveCalc());

// ✅ pass a FUNCTION: runs only on the first render
const [data, setData] = useState(() => expensiveCalc());
```

🕐 State is a Snapshot

```
function handleClick() {
  setCount(count + 1);
  console.log(count);   // still the OLD value! New value appears on the next render
}
```

✅ Key Points

- `useState` returns `[value, setValue]`.
- The argument is the **initial value**, used **only on the first render**.
- Calling the setter triggers a **re-render** with the new value.
- Each `useState` call is an **independent** piece of state; you can use many in one component.
- **Never mutate** state directly; always create a **new** object/array (spread `...`, `map`, `filter`).
- State is a **snapshot**: inside one render, the value never changes, and the update shows up on the **next** render.
- Use the **functional form** `setX(prev => ...)` when the new value depends on the previous one.
- React **batches** multiple updates in one handler into a single re-render.
- Unlike class `setState`, the setter **replaces** the value; it does **not merge** objects.
- If the new value is the **same** as the old (checked with `Object.is`), React skips the re-render.
- **Lazy init**: `useState(() => heavy())` avoids re-running expensive code every render.
- State is **per component instance**; two `<Counter />` have separate counts.

> 💡 **Memory tip:** The value is "what's in the box now"; the setter is "the only way to change what's in the box".
>
> 🎤 **Say it in one line:** "useState lets a component remember a value between renders and re-render when it changes."

1️⃣6️⃣ useEffect

> **Run code that reaches outside React (fetching, timers, subscriptions) after the component renders.**

📖 Explanation

Rendering must stay **pure** (just calculate JSX). Anything else (API calls, timers, event listeners, changing `document.title`) is a **side effect**, and it belongs in `useEffect`. It runs **after** React updates the screen.

🧩 Syntax

```
useEffect(() => {
  // 1. EFFECT: code to run
  return () => {
    // 2. CLEANUP (optional): runs before the next effect and on unmount
  };
}, [dep1, dep2]);   // 3. DEPENDENCY ARRAY: when to re-run
```

🧠 The Dependency Array

Written as	Effect runs…
`useEffect(fn)`	After **every** render
`useEffect(fn, [])`	**Once**, after the first render (mount)
`useEffect(fn, [a, b])`	After mount **and** whenever `a` or `b` change

💻 Code — Timer with Cleanup

```
import { useEffect, useState } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(id);          // cleanup: stop timer on unmount
  }, []);                                    // [] = set up once
```

```
  return <p>{time.toLocaleTimeString()}</p>;
}
```

💻 Code — Fetching Data (the right way)

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();      // lets us cancel the request
    setLoading(true);

    async function load() {                        // effect itself can't be async,
      try {                                        // so define an async fn inside
        const res = await fetch(`/api/users/${userId}`, { signal: controller.signal });
        if (!res.ok) throw new Error("Request failed");
        setUser(await res.json());
        setError(null);
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    load();

    return () => controller.abort();               // cleanup: cancel old request
  }, [userId]);                                    // re-fetch when userId changes

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <h2>{user.name}</h2>;
}
```

💻 Code — Event Listener & Document Title

```
useEffect(() => {
  document.title = `You clicked ${count} times`;      // simple side effect
}, [count]);

useEffect(() => {
  const onResize = () => console.log(window.innerWidth);
  window.addEventListener("resize", onResize);
  return () => window.removeEventListener("resize", onResize);   // always clean up
}, []);
```

⚠️ Common Mistakes

```
// ❌ Infinite loop: sets state on every render, which re-renders, which re-runs the effect...
useEffect(() => { setCount(count + 1); });

// ❌ Missing dependency: uses `userId` but doesn't list it (stale data bug)
useEffect(() => { fetchUser(userId); }, []);

// ❌ Object/array in deps: new reference every render → effect runs every render
useEffect(() => {...}, [{ a: 1 }]);

// ❌ Unnecessary effect: derived data should just be calculated
useEffect(() => { setFullName(first + " " + last); }, [first, last]);
const fullName = first + " " + last;   // ✅ simply calculate it during render
```

✅ Key Points

- `useEffect` runs **after** the component renders and the browser paints.
- It is for **side effects**: fetching data, timers, subscriptions, event listeners, changing the DOM/title.
- **No array** → runs after every render. `[]` → once on mount. `[x]` → when `x` changes.
- **Cleanup**: the function you **return** runs before the next effect and when the component unmounts.
- Always clean up **timers, listeners, subscriptions, and requests** to prevent memory leaks.
- List **every** reactive value (props/state) used inside the effect in the dependency array.
- You **can't** make the effect callback `async`; define an async function **inside** and call it.
- Use `AbortController` (or an `ignore` flag) to avoid **race conditions** when fetching.
- In **dev with StrictMode**, effects run **twice** (mount → cleanup → mount) to test your cleanup. It's normal.
- Setting state inside an effect without correct deps causes an **infinite loop**.
- **Don't use an effect** for things that can be calculated during render, or for responding to user clicks (use event handlers).
- Objects, arrays and functions created in the component are **new every render**, so they make poor dependencies unless memoized.

> 💡 **Memory tip:** The dependency array answers one question: "When should this effect run again?"
>
> 🎤 **Say it in one line:** "useEffect runs side effects after render and re-runs only when its dependencies change, with an optional cleanup."

1️⃣7️⃣ useContext

> Share data with many components without passing props through every level.

📖 Explanation

Prop drilling = passing props through many layers that don't use them. **Context** lets a parent provide a value and any descendant read it directly, however deep.

🔄 Prop Drilling vs Context

```
Prop drilling:  App → Layout → Sidebar → Menu → UserName    (props pass through 3 useless laye

Context:        App (Provider) ·····················► UserName (useContext)
                    └── Layout → Sidebar → Menu     (skipped completely)
```

💻 Code — 3 Steps: Create, Provide, Consume

```jsx
import { createContext, useContext, useState } from "react";

// 1️⃣ CREATE the context (the argument is the default value, used if there is no Provider)
const ThemeContext = createContext("light");

// 2️⃣ PROVIDE a value to everything inside
function App() {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <ThemedButton />;        // no props needed here
}

// 3️⃣ CONSUME with useContext in ANY descendant
function ThemedButton() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button
      className={theme}
      onClick={() => setTheme(theme === "light" ? "dark" : "light")}
```

```
    >
      Current: {theme}
    </button>
  );
}
```

💻 Code — Clean Pattern: Custom Provider + Custom Hook

```
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (u) => setUser(u);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used inside <AuthProvider>");
  return ctx;
}

// Usage anywhere:  const { user, logout } = useAuth();
```

✅ Key Points

- Context solves **prop drilling**.
- Three steps: `createContext` → wrap with `<Context.Provider value={...}>` → read with `useContext(Context)`.
- Any **descendant** of the Provider can read the value, no matter how deep.
- The `createContext(default)` default is used **only when there is no Provider** above.
- When the Provider's `value` **changes**, **every consumer re-renders**.
- Passing a **new object** (`value={{ a, b }}`) each render forces consumers to re-render; wrap it in `useMemo` if it matters.
- Best for **global-ish data**: theme, logged-in user, language, app settings.
- **Not ideal** for fast-changing data (e.g., typing on every keystroke) or huge global state; consider Redux/Zustand.
- Wrap Provider + `useContext` into a **custom provider and custom hook** (`useAuth`) for clean code.
- You can have **multiple contexts** and nest providers.

- A component reads the **nearest** Provider above it.
- In **React 19**, you can write `<ThemeContext value={...}>` without `.Provider`.

> 💡 **Memory tip:** Context is a radio station: the Provider broadcasts, any component can tune in with `useContext`.
>
> 🎤 **Say it in one line:** "Context shares data across the component tree without passing props at every level."

1️⃣8️⃣ useRef

> **Keep a value between renders without causing a re-render, and reach real DOM elements.**

📖 Explanation

`useRef(initial)` returns a plain object `{ current: initial }` that **stays the same across renders**. Changing `ref.current` **does not** re-render the component.

💻 Code — Access a DOM Element

```
import { useRef, useEffect } from "react";

function SearchBox() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus();          // auto-focus when component mounts
  }, []);

  return (
    <>
      <input ref={inputRef} placeholder="Search..." />   {/* attach the ref */}
      <button onClick={() => inputRef.current.select()}>Select text</button>
    </>
  );
}
```

💻 Code — Store a Value (timer id, previous value, render count)

```
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null);            // holds timer id, no re-render needed
```

```
  const start = () => {
    if (intervalRef.current) return;           // already running
    intervalRef.current = setInterval(() => setTime((t) => t + 1), 1000);
  };
  const stop = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  };

  return (
    <>
      <p>{time}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </>
  );
}

// Remember the previous value
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; });   // updates AFTER render
  return ref.current;                          // returns the old value during render
}
```

🧠 useRef vs useState

	useState	useRef
Change triggers re-render?	✅ Yes	❌ No
Value persists between renders?	✅ Yes	✅ Yes
Read/write	Via setter	Directly `ref.current`
Use for	Data shown in the UI	Data **not** shown in the UI, DOM nodes

✅ Key Points

- `useRef(initial)` returns an object `{ current: initial }`.
- The same object is kept for the component's whole life.
- Changing `ref.current` **does NOT re-render**.
- Attach to a JSX element with `ref={myRef}` to get the real DOM node in `myRef.current`.

- The DOM ref is `null` **until after the first render**, so use it in effects or event handlers.
- Common uses: **focus** an input, **scroll** to an element, **measure** size, **play/pause** media.
- Also used to store **timer/interval IDs**, **previous values**, and **mutable values** that shouldn't affect the UI.
- Rule of thumb: if changing it should **update the screen** → `useState`; if not → `useRef`.
- **Don't read or write** `ref.current` **during rendering** (except initial setup); do it in handlers/effects.
- Refs are the **escape hatch** out of React into the DOM; use sparingly.
- Uncontrolled form inputs use refs (see Part 1, Forms).

> 💡 **Memory tip:** A ref is a secret pocket: you can put things in and take them out, and React never notices (no re-render).
>
> 🎤 **Say it in one line:** "useRef holds a mutable value or DOM node across renders without triggering re-renders."

1️⃣9️⃣ useMemo

> **Remember (cache) the result of a calculation and only recompute when dependencies change.**

📖 Explanation

By default, every render re-runs all code in your component. If a calculation is **slow**, `useMemo` stores its result and reuses it until a dependency changes.

💻 Code

```
import { useMemo, useState } from "react";

function ProductList({ products }) {
  const [filter, setFilter] = useState("all");
  const [darkMode, setDarkMode] = useState(false);

  // Only recalculates when `products` or `filter` change.
  // Toggling darkMode re-renders, but does NOT redo the filtering.
  const visible = useMemo(() => {
    console.log("filtering...");
    return products.filter((p) => filter === "all" || p.category === filter);
  }, [products, filter]);

  return (
    <div className={darkMode ? "dark" : ""}>
```

```
      <button onClick={() => setDarkMode(!darkMode)}>Toggle theme</button>
      {visible.map((p) => <li key={p.id}>{p.name}</li>)}
    </div>
  );
}
```

💻 Code — Keeping Object/Array References Stable

```
// New object every render → children wrapped in React.memo would re-render anyway
const options = { sort: "asc" };

// Same object until `sort` changes
const options = useMemo(() => ({ sort }), [sort]);
```

✅ Key Points

- `useMemo(() => compute(), [deps])` returns a **cached value**.
- It **recomputes only** when a dependency changes.
- It caches **values** (numbers, arrays, objects), not functions (that's `useCallback`).
- Good for: **expensive calculations** (sorting, filtering big lists, heavy math).
- Good for: keeping an **object/array reference stable** so `React.memo` children or effect dependencies don't trigger needlessly.
- It's **not free**: it has memory and comparison cost, so **don't wrap everything**.
- Only add it **after noticing a real performance problem** (measure first).
- The cache is a **performance hint, not a guarantee**; React may drop it, so your code must still work without it.
- The function passed must be **pure** and **return** a value.
- Dependencies are compared with `Object.is` (shallow).
- Simple test: *"Is it slow AND running on every render?"* If yes, memoize.
- The newer **React Compiler** can add much of this memoization automatically.

> 💡 **Memory tip:** useMemo is a calculator's memory button: same inputs, so show the saved answer instead of recomputing.
>
> 🎤 **Say it in one line:** "useMemo caches a computed value and recalculates it only when its dependencies change."

2️⃣0️⃣ useCallback

> **Remember (cache) a function so it keeps the same reference between renders.**

📖 Explanation

Functions defined inside a component are **recreated on every render**, so each render produces a **new function reference**. That matters when you pass the function to a memoized child or use it in a dependency array. `useCallback` keeps the same function until its dependencies change.

💻 Code

```
import { useCallback, useState, memo } from "react";

function Parent() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  // ❌ New function every render → memo(Child) re-renders every time anyway
  // const handleAdd = () => setCount((c) => c + 1);

  // ✅ Same function reference across renders
  const handleAdd = useCallback(() => {
    setCount((c) => c + 1);        // functional update → no need to list `count`
  }, []);

  return (
    <>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <p>Count: {count}</p>
      <AddButton onAdd={handleAdd} />   {/* typing in input does NOT re-render this */}
    </>
  );
}

const AddButton = memo(function AddButton({ onAdd }) {
  console.log("AddButton rendered");
  return <button onClick={onAdd}>+1</button>;
});
```

🧠 useMemo vs useCallback

	`useMemo`	`useCallback`
Caches	A **value** (result of a function)	The **function itself**
Equivalent	`useMemo(() => value, deps)`	`useMemo(() => fn, deps)`
Typical use	Heavy calculation	Callback passed to a memoized child

✅ Key Points

- `useCallback(fn, [deps])` returns the **same function reference** until a dependency changes.
- Every render normally creates a **brand-new function**; `useCallback` prevents that.
- It only helps when paired with `React.memo` (child skips re-render if props are equal) or when the function is a **dependency** of `useEffect` / `useMemo`.
- `useCallback` **alone does nothing useful**; a normal child re-renders anyway whenever its parent does.
- `useMemo` **caches a value;** `useCallback` **caches a function.**
- `useCallback(fn, deps)` is the same as `useMemo(() => fn, deps)`.
- Use **functional state updates** (`setX(prev => ...)`) to avoid listing state in the dependencies.
- A wrong or missing dependency leads to a **stale closure** (the function sees old values).
- Don't add it everywhere; use it only when there's a measurable benefit.
- Using it is about **referential equality** (same reference), not about making the function faster.

> 💡 **Memory tip:** useCallback keeps the *same remote control* instead of buying a new one every render.
>
> 🎤 **Say it in one line:** "useCallback memoizes a function so child components relying on its reference don't re-render needlessly."

2️⃣1️⃣ useReducer

> **A more organized way to manage complex state using actions and a reducer function.**

📖 Explanation

Instead of many `setState` calls scattered around, you **describe what happened** by dispatching an **action**, and one **reducer** function decides the **next state**. This is the same pattern Redux uses.

🔄 Flow

```
UI event ──► dispatch({ type: "add" }) ──► reducer(state, action) ──► new state ──► re-rende
```

💻 Code — Counter

```jsx
import { useReducer } from "react";

// Reducer: pure function (state, action) → newState
function reducer(state, action) {
  switch (action.type) {
    case "inc":   return { count: state.count + 1 };
    case "dec":   return { count: state.count - 1 };
    case "reset": return { count: 0 };
    case "add":   return { count: state.count + action.payload };
    default:      throw new Error("Unknown action: " + action.type);
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: "inc" })}>+</button>
      <button onClick={() => dispatch({ type: "dec" })}>-</button>
      <button onClick={() => dispatch({ type: "add", payload: 10 })}>+10</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </>
  );
}
```

💻 Code — Todo List (where it shines)

```jsx
function todoReducer(todos, action) {
  switch (action.type) {
    case "add":
      return [...todos, { id: Date.now(), text: action.text, done: false }];
    case "toggle":
      return todos.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case "delete":
      return todos.filter((t) => t.id !== action.id);
    default:
      return todos;
  }
}

function Todos() {
  const [todos, dispatch] = useReducer(todoReducer, []);
  // dispatch({ type: "add", text: "Learn useReducer" })
  // dispatch({ type: "toggle", id: 5 })
}
```

🧠 useState vs useReducer

	`useState`	`useReducer`
Best for	Simple, independent values	Complex or related state, many actions
Update logic	Spread across handlers	Centralized in one reducer
Testing	Harder	Easy (reducer is a pure function)
Code size	Less for simple cases	More boilerplate

✅ Key Points

- `useReducer(reducer, initialState)` returns `[state, dispatch]`.
- `dispatch(action)` sends a plain object (usually `{ type, payload }`) describing **what happened**.
- The **reducer** is a **pure function**: `(state, action) => newState`.
- Reducers must be **pure**: no API calls, no random values, no mutating the old state.
- **Return a new object/array**; never mutate the `state` argument.
- Great when **many actions** update **related** pieces of state (forms, carts, todos).
- Puts all update logic in **one predictable, testable place**.
- `dispatch` has a **stable identity**, so it's safe to pass down or leave out of dependency arrays.
- Pair it with **Context** (`useReducer` + `createContext`) for a mini-Redux without libraries.
- Optional 3rd argument: an **init function** for lazy initial state.
- Always include a `default` case that returns the state (or throws).
- Rule of thumb: start with `useState`; switch to `useReducer` when state logic gets tangled.

> 💡 **Memory tip:** Dispatch is filling a request form; the reducer is the clerk who decides the new state.
>
> 🎤 **Say it in one line:** "useReducer manages complex state by dispatching actions to a pure reducer function."

2️⃣2️⃣ Custom Hooks

> **Your own reusable hook: a function starting with `use` that calls other hooks.**

📖 Explanation

If two components share the same stateful logic, **extract it into a custom hook**. It's just a function that uses built-in hooks inside. Custom hooks share **logic**, not **state**: each component that calls it gets its **own separate copy** of the state.

💻 Code — `useToggle`

```
function useToggle(initial = false) {
  const [value, setValue] = useState(initial);
  const toggle = useCallback(() => setValue((v) => !v), []);
  return [value, toggle];
}

// Usage
const [isOpen, toggleOpen] = useToggle();
```

💻 Code — `useLocalStorage`

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage (works exactly like useState, but persists!)
const [theme, setTheme] = useLocalStorage("theme", "light");
```

💻 Code — `useFetch`

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);

    fetch(url, { signal: controller.signal })
      .then((res) => {
        if (!res.ok) throw new Error("Request failed");
```

```
        return res.json();
      })
      .then((json) => { setData(json); setError(null); })
      .catch((err) => { if (err.name !== "AbortError") setError(err.message); })
      .finally(() => setLoading(false));

    return () => controller.abort();
  }, [url]);

  return { data, loading, error };
}

// Usage
const { data, loading, error } = useFetch("https://api.example.com/users");
```

💻 Code — `useDebounce` (wait until the user stops typing)

```
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);       // reset the timer on every change
  }, [value, delay]);

  return debounced;
}

// Usage: call the API only when typing pauses
const [query, setQuery] = useState("");
const debouncedQuery = useDebounce(query, 500);
useEffect(() => { search(debouncedQuery); }, [debouncedQuery]);
```

✅ Key Points

- A custom hook is a **function whose name starts with** `use` and calls other hooks.
- The `use` **prefix is required**, so React and ESLint can enforce the Rules of Hooks.
- It **extracts and reuses logic** across components, removing duplicated code.
- Custom hooks share **logic, not state**: each call creates **independent** state.
- It can **return anything**: a value, an array `[value, fn]`, or an object `{ data, loading }`.
- Return an **array** for 1–2 items you'll rename (like `useState`); return an **object** for many items.
- It can take **arguments** (`useFetch(url)`), which can be reactive.
- Usually built from `useState`, `useEffect`, `useRef`, `useCallback`, etc.
- It must follow the **Rules of Hooks** itself (top level, only inside components or other hooks).

- Common examples: `useFetch`, `useForm`, `useLocalStorage`, `useDebounce`, `useToggle`, `useWindowSize`, `useAuth`.
- Keep each hook focused on **one job**; name it by what it does, not when it runs.
- Don't create custom hooks for **lifecycle names** like `useMount`; prefer clear purposes.

> 💡 **Memory tip:** A custom hook is a recipe you can reuse in any kitchen. Each cook still makes their own dish (state).
>
> 🎤 **Say it in one line:** "A custom hook packages reusable stateful logic into a function that starts with `use`."

2️⃣3️⃣ Other Hooks

> **Five extra hooks you should recognize and be able to explain.**

🧰 `useLayoutEffect` — run before the browser paints

```
import { useLayoutEffect, useRef, useState } from "react";

function Tooltip() {
  const ref = useRef(null);
  const [height, setHeight] = useState(0);

  // Measure the DOM BEFORE the screen is painted → no visible flicker
  useLayoutEffect(() => {
    setHeight(ref.current.getBoundingClientRect().height);
  }, []);

  return <div ref={ref}>Tooltip (height: {height}px)</div>;
}
```

- Same signature as `useEffect`, but runs **synchronously after DOM updates and before paint**.
- Use only for **measuring/adjusting layout**; otherwise prefer `useEffect` (it doesn't block painting).

🧰 `useId` — unique, stable IDs

```
import { useId } from "react";

function EmailField() {
  const id = useId();                    // e.g. ":r1:"
  return (
```

```
    <>
      <label htmlFor={id}>Email</label>
      <input id={id} type="email" />
    </>
  );
}
```

- Generates a **unique ID** that matches on server and client.
- Great for linking `label` ↔ `input` and accessibility attributes (`aria-*`).
- **Don't use it as a list** `key`.

🧰 `useTransition` — mark an update as non-urgent

```
import { useState, useTransition } from "react";

function Search({ bigList }) {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState(bigList);
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    setQuery(e.target.value);                    // urgent: input stays responsive
    startTransition(() => {
      setResults(bigList.filter((x) => x.includes(e.target.value)));   // can wait
    });
  }

  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <p>Updating...</p>}
      <ul>{results.map((r) => <li key={r}>{r}</li>)}</ul>
    </>
  );
}
```

- Returns `[isPending, startTransition]`.
- Updates inside `startTransition` are **low priority** and can be **interrupted** by urgent ones (like typing).
- Keeps the UI **responsive** during heavy re-renders.

🧰 `useDeferredValue` — delay updating a value

```
import { useDeferredValue, useState } from "react";

function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);   // lags behind during heavy renders
```

```
  const isStale = query !== deferredQuery;
  return <div style={{ opacity: isStale ? 0.5 : 1 }}>{/* heavy list using deferredQuery */}</div>
}
```

- Gives you a **"lagging copy"** of a value that updates after urgent work.
- Use it when **you don't control the code that sets the state** (e.g., a prop); use `useTransition` when you do.

🧰 `useImperativeHandle` — control what a parent can do with a ref

```
import { useImperativeHandle, useRef } from "react";

// React 19: `ref` is a normal prop. (React 18 needed forwardRef.)
function FancyInput({ ref }) {
  const inputRef = useRef(null);

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),     // expose ONLY these methods
    clear: () => (inputRef.current.value = ""),
  }));

  return <input ref={inputRef} />;
}

// Parent: const r = useRef(); <FancyInput ref={r} />;  r.current.focus();
```

- Customizes the **instance value** exposed to a parent through a `ref`.
- Rarely needed; use it when a parent must call methods (`focus`, `reset`) on a child.

✅ Key Points

- `useLayoutEffect`: runs **before paint**, for measuring layout; **blocks** painting, so use sparingly.
- `useEffect` is the default; reach for `useLayoutEffect` only to **fix visual flicker**.
- `useId`: generates **unique stable IDs** for accessibility; never use as a `key`.
- `useTransition`: wraps **non-urgent state updates** so the UI stays responsive; gives `isPending`.
- `useDeferredValue`: returns a **deferred (delayed) version** of a value.
- `useTransition` wraps the **update**; `useDeferredValue` wraps the **value**.
- `useImperativeHandle`: exposes a **limited set of methods** to a parent via `ref`.
- The last three are **performance/advanced** tools; most apps rarely need them.
- All of these still follow the **Rules of Hooks**.

> 💡 **Memory tip:** useLayoutEffect = "before paint", useId = "name tag", useTransition/useDeferredValue = "this can wait", useImperativeHandle = "remote with

> fewer buttons".
>
> 🎤 **Say it in one line:** "These extra hooks cover layout timing, unique IDs, prioritizing updates, and controlling refs."

📝 Part 2 Cheat Sheet

Hook	Purpose	Use it when…
`useState`	Store changing data	You need simple local state
`useEffect`	Run side effects after render	Fetching, timers, subscriptions, listeners
`useContext`	Read shared data from a Provider	Avoiding prop drilling (theme, auth)
`useRef`	Persist a value / access a DOM node	Focus, timer IDs, previous value (no re-render)
`useMemo`	Cache a computed **value**	An expensive calculation re-runs needlessly
`useCallback`	Cache a **function** reference	Passing callbacks to `React.memo` children
`useReducer`	Manage complex state via actions	Many related actions update one state
Custom hook	Reuse stateful logic	Same logic appears in 2+ components
`useLayoutEffect`	Effect before paint	Measuring DOM, avoiding flicker
`useId`	Unique ID	Label ↔ input linking
`useTransition`	Mark update as low priority	Heavy updates freezing typing
`useDeferredValue`	Lagging copy of a value	Heavy render from a prop/value you don't control

Hook	Purpose	Use it when…
`useImperativeHandle`	Limit what a ref exposes	Parent needs `focus()` / `reset()` on a child

🧭 Dependency Array Quick Table

Code	Runs
`useEffect(fn)`	Every render
`useEffect(fn, [])`	Once on mount
`useEffect(fn, [x])`	On mount + when `x` changes
`return () => {...}`	Cleanup: before next run and on unmount

🔥 Top 8 Quick-Fire Questions

1. **Why can't hooks be in conditions or loops?** → React tracks hooks by call order; changing the order breaks the state mapping.
2. `useState` **vs** `useRef`**?** → State changes re-render; ref changes don't.
3. **What does an empty dependency array mean?** → Run the effect once, after the first render.
4. **Why is cleanup needed in** `useEffect`**?** → To stop timers/listeners/requests and avoid leaks and bugs.
5. `useMemo` **vs** `useCallback`**?** → useMemo caches a value; useCallback caches a function.
6. **When use** `useReducer` **over** `useState`**?** → Complex or related state with many update actions.
7. **Do custom hooks share state?** → No, they share logic; each caller gets its own state.
8. **Why does my effect run twice in development?** → StrictMode intentionally mounts, unmounts, mounts to check your cleanup.

✅ **Part 2 complete.** Next: **Part 3 — Routing & Data** (Topics 24–27).

⚛️ React Revision Course — Part 1: Fundamentals

> **Topics 1–13** · Target: ~16 pages · Simple English · Revise from zero
>
> **How to use this file:** Read the *explanation*, run the *code*, then cover the page and try to say the *key points* out loud. If you can explain 7 out of 10 points, you've absorbed the topic.

📑 Contents

#	Topic	#	Topic
1	What is React	8	Conditional Rendering
2	Setup & Project Structure	9	Lists & Keys
3	JSX	10	Forms
4	Components	11	Lifting State Up
5	Props	12	Component Lifecycle
6	State Basics	13	Styling
7	Event Handling		**Cheat Sheet** (end)

1️⃣ What is React

> **A JavaScript library for building user interfaces out of small, reusable pieces called components.**

📖 Explanation

React was created by Meta (Facebook) and released in 2013. Instead of manually changing the page with `document.getElementById(...)`, you **describe what the UI should look like** for the

current data, and React updates the real page for you. You change the data, React updates the screen.

🔄 How React Updates the Screen

```
Data (state/props) changes
        |
        ▼
React builds a NEW Virtual DOM (a light JS copy of the UI)
        |
        ▼
React compares new vs old Virtual DOM   ← this is "diffing" / reconciliation
        |
        ▼
React updates ONLY the parts of the real DOM that changed
```

💻 Code — Vanilla JS vs React

```
// ❌ Vanilla JS (imperative: you give every step)
const btn = document.getElementById("btn");
let count = 0;
btn.addEventListener("click", () => {
  count++;
  btn.textContent = "Clicked " + count + " times"; // manual DOM update
});

// ✅ React (declarative: you describe the result)
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

🧠 Library vs Framework

	Library (React)	Framework (Angular)
Control	**You** call the library	Framework calls **your** code
Scope	Only the UI layer	Routing, forms, HTTP, everything
Freedom	You choose router, state tool, etc.	Opinionated, one official way

✅ Key Points

- React is a **library**, not a full framework. It only handles the UI (the "View").
- It is **component-based**: the UI is split into small, reusable pieces.
- It is **declarative**: you say *what* to show, not *how* to change the DOM step by step.
- The **Virtual DOM** is a lightweight JavaScript copy of the real DOM.
- **Reconciliation** = React comparing old and new Virtual DOM to find the minimum changes.
- Data flows **one way**: parent → child (via props). This makes bugs easier to trace.
- React apps are usually **SPAs** (Single Page Applications): one HTML page, content swaps without full reloads.
- React doesn't include routing or HTTP; you add `react-router`, `axios`, etc.
- It uses **JSX**, a syntax that mixes HTML-like markup into JavaScript.
- Huge ecosystem and job market, which is why it is so widely used.

> 💡 **Memory tip:** React is like a smart painter. You tell it the final picture; it only repaints the spots that changed.
>
> 🎤 **Say it in one line:** "React is a component-based library that updates the UI efficiently using a virtual DOM."

2️⃣ Setup & Project Structure

> **Create a project with Vite, understand every file, and know the 4 scripts you'll use daily.**

📖 Explanation

Vite is a fast build tool that creates and runs a React project. (The older tool, Create React App, is no longer recommended.) You need **Node.js** installed first, because `npm` comes with it.

💻 Commands (go to your projects folder first)

```
cd C:\Users\YourName\Projects
npm create vite@latest my-app -- --template react
```

```
cd C:\Users\YourName\Projects\my-app
npm install
```

```
cd C:\Users\YourName\Projects\my-app
npm run dev
```

Open the URL shown in the terminal (usually `http://localhost:5173`).

📁 Project Structure

```
my-app/
├── node_modules/      ← installed packages (never edit, never upload)
├── public/            ← static files served as-is (favicon, images)
├── src/               ← YOUR code lives here
│   ├── assets/        ← images, fonts used by components
│   ├── App.jsx        ← root component
│   ├── App.css
│   ├── main.jsx       ← entry point (connects React to HTML)
│   └── index.css      ← global styles
├── index.html         ← the ONE html page (has <div id="root">)
├── package.json       ← dependencies + scripts
└── vite.config.js     ← Vite settings
```

💻 Code — `main.jsx` (the entry point)

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

// Find <div id="root"> in index.html and render <App /> inside it
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

💻 Code — `App.jsx`

```
function App() {
  return <h1>Hello React 👋</h1>;
}

export default App;
```

🛠️ Scripts in `package.json`

Command	What it does
`npm run dev`	Starts the dev server with hot reload

Command	What it does
`npm run build`	Creates the optimized production build in `dist/`
`npm run preview`	Previews the production build locally
`npm run lint`	Checks code for problems (ESLint)

✅ Key Points

- You need **Node.js** installed; it provides `npm`.
- **Vite** is the modern, fast way to start a React project.
- `index.html` has a single `<div id="root">`; React renders the whole app inside it.
- `main.jsx` is the **entry point**: it calls `createRoot(...).render(<App />)`.
- `App.jsx` is the **root component**; everything else is nested inside it.
- All your code goes in the `src/` folder.
- `package.json` lists dependencies and scripts.
- **Never edit or upload** `node_modules`; `npm install` recreates it.
- **StrictMode** is a dev-only helper that runs extra checks (and runs effects twice in dev).
- Component files use the `.jsx` extension.

> 💡 **Memory tip:** `index.html` is the empty stage, `main.jsx` is the stagehand that sets it up, `App.jsx` is the first actor.
>
> 🎤 **Say it in one line:** "Vite scaffolds the project; main.jsx mounts the App component into the root div."

3️⃣ JSX

> **JavaScript XML: HTML-like syntax inside JavaScript that describes what the UI looks like.**

📖 Explanation

JSX is **not HTML**. It's syntax sugar that gets converted into `React.createElement(...)` calls. Browsers can't read JSX, so Vite converts it to plain JavaScript for you.

💻 Code

```jsx
function Profile() {
  const name = "Abdul";
  const age = 20;
  const isStudent = true;
  const styleBox = { backgroundColor: "lightblue", padding: "10px" };

  return (
    <>                                          {/* Fragment: one root, no extra div */}
      <h1 className="title">Hello, {name}!</h1>  {/* {} runs JS */}
      <p>Next year you will be {age + 1}.</p>    {/* any expression */}
      <p>{isStudent ? "Student" : "Not a student"}</p>
      <div style={styleBox}>Styled with an object</div>
      <img src="photo.jpg" alt="me" />          {/* self-close every tag */}
      <input type="text" disabled />
    </>
  );
}
```

🔄 What JSX Becomes

```jsx
<h1 className="title">Hi</h1>
// is converted to:
React.createElement("h1", { className: "title" }, "Hi");
```

✅ Key Points

- JSX = HTML-like syntax **inside JavaScript**, converted to `React.createElement`.
- A component must return **one root element**. Wrap siblings in a `<div>` or a **Fragment** `<>...</>`.
- **Every tag must be closed**: `<img />`, `<input />`, `<br />`.
- Use `className` instead of `class`, and `htmlFor` instead of `for` (these are reserved JS words).
- Attributes are **camelCase**: `onClick`, `tabIndex`, `maxLength`.
- Use `{ }` to insert any JavaScript **expression** (variables, math, function calls, ternary).
- You **cannot** put statements (`if`, `for`) inside `{ }`; use ternary or compute before `return`.
- The inline `style` takes an **object** with camelCase keys: `style={{ fontSize: "20px" }}` (double braces = braces for JS + braces for the object).
- Comments inside JSX look like `{/* comment */}`.
- Component names **must start with a capital letter**; lowercase means a normal HTML tag.

> 💡 **Memory tip:** `{ }` is a "window" from the markup back into JavaScript.
>
> 🎤 **Say it in one line:** "JSX lets me write UI markup in JavaScript; it compiles to createElement calls."

4️⃣ Components

> **Reusable, independent pieces of UI. React apps are trees of components.**

📖 Explanation

A component is just a **JavaScript function that returns JSX**. Build small components and combine (compose) them into bigger ones, like LEGO bricks.

🌳 Component Tree

```
App
├── Header
│   └── Logo
├── Main
│   ├── ProductCard
│   ├── ProductCard
│   └── ProductCard
└── Footer
```

💻 Code

```jsx
// Header.jsx
function Header() {
  return <header><h1>My Shop</h1></header>;
}
export default Header;                  // default export

// ProductCard.jsx
export function ProductCard() {         // named export
  return <div className="card">Product</div>;
}

// App.jsx — composition: using components like HTML tags
import Header from "./Header";
import { ProductCard } from "./ProductCard";

function App() {
  return (
    <div>
      <Header />
      <ProductCard />
      <ProductCard />   {/* reuse as many times as needed */}
```

```
    </div>
  );
}
export default App;
```

✅ Key Points

- A component = a **function that returns JSX** (modern React uses function components).
- Name must start with a **Capital letter** (`Header`, not `header`).
- Use it like a tag: `<Header />`.
- **Composition**: build big UIs by nesting small components.
- Components are **reusable**; each use creates an independent copy with its own state.
- Convention: **one component per file**, and the file name matches the component name.
- **Default export**: one per file, imported without braces: `import Header from "./Header"`.
- **Named export**: many per file, imported with braces: `import { ProductCard } from "./ProductCard"`.
- Never define a component **inside** another component (it resets state on every render).
- Components should be **pure**: same inputs (props) → same output (JSX).

> 💡 **Memory tip:** Components are LEGO bricks: small, reusable, snap together.
>
> 🎤 **Say it in one line:** "A component is a reusable function that returns JSX describing part of the UI."

5️⃣ Props

> **Inputs passed from a parent to a child component, like function arguments.**

📖 Explanation

Props (properties) let a parent send data to a child. They flow **one way: down**. The child can **read** props but must **never change** them.

🔄 Flow

```
Parent  ──(props)──►  Child
 owns data             only reads data
```

💻 Code

```jsx
// Child: receives props and destructures them, with a default value
function UserCard({ name, age = 18, isOnline, children }) {
  return (
    <div>
      <h2>{name} ({age})</h2>
      <p>{isOnline ? "🟢 Online" : "⚪ Offline"}</p>
      {children}                          {/* whatever is placed between the tags */}
    </div>
  );
}

// Parent: passes props like HTML attributes
function App() {
  const user = { name: "Sara", age: 22, isOnline: true };

  return (
    <>
      <UserCard name="Ali" isOnline={false} />          {/* age uses default 18 */}
      <UserCard {...user}>                              {/* spread an object */}
        <button>Message</button>                        {/* becomes children */}
      </UserCard>
    </>
  );
}
```

✅ Key Points

- Props are passed like HTML attributes: `<Card title="Hi" />`.
- Strings use quotes; **everything else uses braces**: `age={22}`, `isOn={true}`, `items={[1,2]}`.
- A component receives **one object** (`props`); **destructure** it: `function Card({ title })`.
- Props are **read-only (immutable)**. Never write `props.name = "x"`.
- Give **default values** in destructuring: `{ age = 18 }`.
- `children` is a special prop containing whatever is nested between the opening and closing tags.
- You can pass **anything**: strings, numbers, arrays, objects, **functions**, even JSX.
- Passing a **function** as a prop is how a child talks back to its parent.
- **Spread** an object into props with `{...user}`.
- When props change, the component **re-renders** automatically.

> 💡 **Memory tip:** Props = function arguments for components. Parents give, children only receive.
>
> 🎤 **Say it in one line:** "Props are read-only inputs passed from parent to child to customize a component."

6️⃣ State Basics

> **Data that belongs to a component and changes over time. Changing it re-renders the UI.**

📖 Explanation

A normal variable resets every time the component function runs again, and changing it does **not** update the screen. **State** is memory that React keeps between renders, and updating it **triggers a re-render**. (Full `useState` details come in Part 2.)

💻 Code — Why a normal variable fails

```
import { useState } from "react";

// ❌ Broken: variable changes but UI never updates
function BadCounter() {
  let count = 0;
  return <button onClick={() => { count++; console.log(count); }}>{count}</button>;
}

// ✅ Works: state update triggers re-render
function GoodCounter() {
  const [count, setCount] = useState(0);   // [currentValue, updaterFunction]
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

🔄 What Happens on Click

```
Click → setCount(1) → React schedules re-render
      → function runs again with count = 1 → new JSX → screen updates
```

🧠 Props vs State

	Props	State
Owned by	Parent	The component itself
Can the component change it?	❌ No (read-only)	✅ Yes (via setter)
Purpose	Configure a component	Remember changing data

	Props	State
Change causes re-render?	✅ Yes	✅ Yes

✅ Key Points

- **State** = data a component remembers between renders.
- A normal variable **loses its value** on re-render and **doesn't** update the UI.
- `useState(initial)` returns `[value, setValue]`.
- **Never** change state directly (`count = 5`); always use the setter.
- Calling the setter **schedules a re-render** with the new value.
- State is **local and private** to each component instance.
- State updates are **batched and asynchronous**: the new value appears on the next render, not on the next line.
- Rule of thumb: if changing it should update the UI → **state**. If not → a normal variable or ref.
- Keep state **minimal**: don't store values you can calculate from other state.
- Props flow down; state lives where it's needed (and can be lifted up, see Topic 11).

> 💡 **Memory tip:** State is the component's notebook. A normal variable is a sticky note that gets thrown away every render.
>
> 🎤 **Say it in one line:** "State is data a component remembers; updating it re-renders the component."

7️⃣ Event Handling

> **Respond to user actions like clicks, typing and form submits.**

📖 Explanation

React events are written in **camelCase** and you pass a **function**, not a string. React wraps browser events in a **SyntheticEvent**, which works the same in all browsers.

💻 Code

```
import { useState } from "react";

function EventDemo() {
  const [text, setText] = useState("");
```

```
  function handleClick() {
    alert("Clicked!");
  }

  function handleDelete(id) {
    console.log("Deleting", id);
  }

  function handleSubmit(e) {
    e.preventDefault();                // stop page reload
    console.log("Submitted:", text);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={(e) => setText(e.target.value)} />

      {/* ✅ pass the function (no parentheses) */}
      <button type="button" onClick={handleClick}>Click</button>

      {/* ✅ need an argument? wrap in an arrow function */}
      <button type="button" onClick={() => handleDelete(5)}>Delete</button>

      {/* ❌ WRONG: runs immediately during render */}
      {/* <button onClick={handleClick()}>Bad</button> */}

      <button type="submit">Submit</button>
    </form>
  );
}
```

📋 Common Events

Event	Fires when
`onClick`	Element is clicked
`onChange`	Input value changes
`onSubmit`	Form is submitted
`onKeyDown`	A key is pressed
`onMouseEnter` / `onMouseLeave`	Mouse enters / leaves
`onFocus` / `onBlur`	Element gains / loses focus

✅ Key Points

- Events are **camelCase**: `onClick`, not `onclick`.
- Pass a **function reference**: `onClick={handleClick}`, **not** `onClick={handleClick()}`.
- To pass arguments, wrap in an **arrow function**: `onClick={() => handleDelete(id)}`.
- The handler receives an **event object** `e` (a SyntheticEvent).
- `e.preventDefault()` stops default behavior (e.g., form reload, link navigation).
- `e.stopPropagation()` stops the event from bubbling up to parents.
- `e.target.value` gives the current value of an input.
- Handlers are usually named `handleSomething`; handler props are named `onSomething`.
- You can pass handlers **as props** so children can notify parents.
- Use `type="button"` on non-submit buttons inside forms to avoid accidental submits.

> 💡 **Memory tip:** Pass the function, don't call it. Parentheses mean "run now".
>
> 🎤 **Say it in one line:** "React handles events with camelCase props that take function references, using synthetic events."

8️⃣ Conditional Rendering

> **Show different UI depending on a condition.**

📖 Explanation

React has no special syntax for this; you use normal JavaScript (`if`, ternary, `&&`) to decide what JSX to return.

💻 Code

```
function Dashboard({ isLoggedIn, isLoading, unread, role }) {
  // 1️⃣ Early return: handle special cases first
  if (isLoading) return <p>Loading...</p>;

  return (
    <div>
      {/* 2️⃣ Ternary: either/or */}
      {isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please log in</h1>}

      {/* 3️⃣ && : show or nothing */}
      {unread > 0 && <p>You have {unread} new messages</p>}

      {/* 4️⃣ Storing JSX in a variable */}
      {role === "admin" ? <AdminPanel /> : null}   {/* null renders nothing */}
```

```
    </div>
  );
}
```

⚠️ The `&&` Trap

```
{count && <p>Items: {count}</p>}      // ❌ if count is 0, it renders "0" on screen!
{count > 0 && <p>Items: {count}</p>}  // ✅ always use a real boolean condition
```

✅ Key Points

- Use plain JavaScript to choose what to render.
- `if/else` goes **outside** the JSX (before `return`); it can't go inside `{ }`.
- **Ternary** `cond ? A : B` works inside JSX for either/or.
- `&&` `cond && <X />` shows X only when the condition is true.
- **Early return** (`if (loading) return <Spinner />`) keeps components clean.
- Returning `null` renders nothing.
- `false`, `null`, `undefined` render nothing, but `0` **renders as "0"**.
- Always use a **boolean condition** with `&&` (`count > 0 &&`).
- Avoid deeply nested ternaries; extract into a variable or small component.
- Typical uses: loading states, error messages, logged-in vs guest, permissions.

> 💡 **Memory tip:** `? :` for two options, `&&` for "show or hide", early `return` for special cases.
>
> 🎤 **Say it in one line:** "I use JavaScript conditions like ternary, && and early returns to control what renders."

9️⃣ Lists & Keys

> **Render many items from an array using `map()`, and give each a unique key.**

📖 Explanation

To show a list, transform an array into an array of JSX with `.map()`. Each item needs a `key` so React can tell which items were added, removed or moved.

💻 Code

```
function TodoList() {
  const todos = [
    { id: 101, text: "Learn JSX", done: true },
```

```
    { id: 102, text: "Learn Props", done: false },
    { id: 103, text: "Learn State", done: false },
  ];

  const pending = todos.filter((t) => !t.done);   // filter first, then map

  return (
    <ul>
      {pending.map((todo) => (
        <li key={todo.id}>{todo.text}</li>   // key on the OUTERMOST element
      ))}
    </ul>
  );
}
```

⚠️ Index as Key — Problem

```
{items.map((item, index) => <li key={index}>{item}</li>)}
// ❌ If items are reordered/deleted, indexes shift → wrong state/inputs get mixed up
// ✅ Use a stable unique id (from database or generated once)
```

✅ Key Points

- Use `array.map()` to turn data into JSX elements.
- Every item needs a `key` prop, placed on the **outermost** element returned from `map`.
- Keys must be **unique among siblings** (not globally) and **stable** (don't change between renders).
- Best key: a **unique id** from your data (database id).
- **Avoid** `index` **as key** if the list can be reordered, filtered or edited.
- **Never use** `Math.random()` as a key (it changes every render, so everything remounts).
- Keys are **not passed as props**; the child can't read `props.key`.
- Without keys React warns and can update the UI incorrectly or slowly.
- Chain `filter` (remove) and `sort` (order, on a copy) before `map`.
- Always render an **empty state** message when the list is empty.

> 💡 **Memory tip:** The key is the item's ID card, so React can recognize it even when the line reorders.
>
> 🎤 **Say it in one line:** "I render lists with map and give each item a stable unique key so React can track it."

🔟 Forms

> Collect user input. Controlled components keep input values in React state.

📖 Explanation

Controlled component: React state is the single source of truth. The input's `value` comes from state, and every change updates state. **Uncontrolled component:** the DOM keeps the value, and you read it when needed (with a `ref`).

🔄 Controlled Input Cycle

```
User types → onChange fires → setState(new value) → re-render → input shows state value
```

💻 Code — Controlled Form with Multiple Inputs

```
import { useState } from "react";

function SignupForm() {
  // One state OBJECT for all fields
  const [form, setForm] = useState({
    name: "", email: "", gender: "male", city: "lahore", agree: false, bio: "",
  });
  const [error, setError] = useState("");

  // ONE handler for all fields (uses the input's name attribute)
  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm({
      ...form,                                    // keep other fields
      [name]: type === "checkbox" ? checked : value // [name] = dynamic key
    });
  }

  function handleSubmit(e) {
    e.preventDefault();
    if (!form.name.trim() || !form.email.includes("@")) {
      setError("Enter a valid name and email");
      return;
    }
    setError("");
    console.log("Submitted:", form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name" />
      <input name="email" type="email" value={form.email} onChange={handleChange} />
```

```
      <label>
        <input type="radio" name="gender" value="male"
               checked={form.gender === "male"} onChange={handleChange} /> Male
      </label>
      <label>
        <input type="radio" name="gender" value="female"
               checked={form.gender === "female"} onChange={handleChange} /> Female
      </label>

      <select name="city" value={form.city} onChange={handleChange}>
        <option value="lahore">Lahore</option>
        <option value="bahawalpur">Bahawalpur</option>
      </select>

      <textarea name="bio" value={form.bio} onChange={handleChange} />

      <label>
        <input type="checkbox" name="agree" checked={form.agree} onChange={handleChange} />
        I agree
      </label>

      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit" disabled={!form.agree}>Sign up</button>
    </form>
  );
}
```

💻 Code — Uncontrolled (with ref)

```
import { useRef } from "react";

function QuickSearch() {
  const inputRef = useRef(null);
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(inputRef.current.value);   // read value only when needed
  };
  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} defaultValue="" />   {/* defaultValue, not value */}
      <button>Search</button>
    </form>
  );
}
```

🧠 Controlled vs Uncontrolled

	Controlled	Uncontrolled
Value stored in	React state	The DOM
Read value with	State variable	`ref.current.value`
Instant validation	✅ Easy	❌ Harder
Best for	Most forms	Simple forms, file inputs

✅ Key Points

- **Controlled input** = `value={state}` + `onChange={update state}`.
- If you set `value` without `onChange`, the input becomes **read-only**.
- Always call `e.preventDefault()` in `onSubmit` to stop the page reload.
- Use **one state object** and **one handler** with `name` attributes for many fields.
- `[e.target.name]: value` (computed key) updates the right field dynamically.
- **Checkbox** uses `checked` (not `value`); get it from `e.target.checked`.
- **Radio**: each has the same `name`, and `checked` compares state to its value.
- `<select>` and `<textarea>` use `value` in React (not `selected` or inner text).
- **Uncontrolled** inputs use `ref` and `defaultValue`; file inputs are always uncontrolled.
- Basic **validation**: check fields in `handleSubmit`, store messages in an `error` state.
- Spread `...form` to keep other fields when updating one (state updates don't merge).

> 💡 **Memory tip:** Controlled = React holds the steering wheel. Uncontrolled = the browser drives, you peek later.
>
> 🎤 **Say it in one line:** "A controlled form stores input values in state so React is the single source of truth."

1️⃣1️⃣ Lifting State Up

> **When two components need the same data, move the state to their closest common parent.**

📖 Explanation

Siblings can't talk to each other directly. Put the state in the **parent**, pass the **value down** as a prop and pass the **setter (or handler) down** so children can request changes.

🔄 Flow

```
          Parent  (owns state: temperature)
         ▲   |  ▲   |
onChange |   |  |   | value + handler as props
         |   ▼  |   ▼
 Celsius Input    Fahrenheit Display
```

💻 Code

```jsx
import { useState } from "react";

function Parent() {
  const [text, setText] = useState("");          // state lives here

  return (
    <>
      <InputBox value={text} onTextChange={setText} />   {/* child 1 changes it */}
      <Preview value={text} />                           {/* child 2 reads it */}
    </>
  );
}

function InputBox({ value, onTextChange }) {
  return <input value={value} onChange={(e) => onTextChange(e.target.value)} />;
}

function Preview({ value }) {
  return <p>You typed: {value}</p>;
}
```

✅ Key Points

- Siblings **cannot share state directly**; the parent must hold it.
- Move state to the **closest common ancestor** of the components that need it.
- Pass **data down** as props and **changes up** via function props (callbacks).
- This keeps a **single source of truth**, so there are no out-of-sync copies.
- Children become "**controlled**" by the parent: they just display and report.
- The data flow stays **one-way (down)**; callbacks are how events travel **up**.
- Don't lift state higher than necessary: keep state as **local** as possible.
- If lifting goes through many levels (**prop drilling**), use **Context** (Part 2) instead.
- Common examples: search box + results list, filter + product list, form + preview.
- Name callback props `onSomething` to make the direction clear.

> 💡 **Memory tip:** Two kids arguing about a toy? Give it to the parent. Both can see it.

> 🎤 **Say it in one line:** "I lift state to the nearest common parent so sibling components stay in sync."

1️⃣2️⃣ Component Lifecycle

> **The three life phases of a component: mount, update, unmount.**

📖 Explanation

Every component is **born** (mounted), **changes** (updated) and **dies** (unmounted). In class components these phases had named methods. In function components, `useEffect` covers all of them (details in Part 2).

🔄 The Three Phases

```
MOUNT                    UPDATE                     UNMOUNT
(added to screen)   →    (props/state change)   →   (removed from screen)
first render             re-render                  cleanup
```

🧠 Class vs Hooks Mapping

Phase	Class method	Function component
Mount	`componentDidMount`	`useEffect(() => {...}, [])`
Update	`componentDidUpdate`	`useEffect(() => {...}, [value])`
Unmount	`componentWillUnmount`	`return () => {...}` (cleanup) inside `useEffect`

💻 Code

```jsx
import { useEffect, useState } from "react";

function Timer() {
  const [seconds, setSeconds] = useState(0);

  // MOUNT: runs once after first render
  useEffect(() => {
    console.log("Mounted ✅");
    const id = setInterval(() => setSeconds((s) => s + 1), 1000);
```

```
    // UNMOUNT: cleanup runs when removed
    return () => {
      console.log("Unmounted ❌");
      clearInterval(id);
    };
  }, []);

  // UPDATE: runs after mount AND every time `seconds` changes
  useEffect(() => {
    console.log("Seconds changed:", seconds);
  }, [seconds]);

  return <p>{seconds}s</p>;
}

// Parent toggles it to see mount/unmount:
// {show && <Timer />}
```

✅ Key Points

- **Mounting** = component appears on screen for the first time.
- **Updating** = it re-renders because **state or props changed** (or the parent re-rendered).
- **Unmounting** = component is removed from the screen.
- **Rendering** (calculating JSX) is different from **committing** (updating the DOM) and from **effects** (side effects after painting).
- `useEffect` with `[]` → runs on **mount** only.
- `useEffect` with `[dep]` → runs on mount and **when** `dep` **changes**.
- The function **returned** from `useEffect` is the **cleanup**, which runs on unmount (and before the effect re-runs).
- **Always clean up** timers, event listeners and subscriptions to avoid memory leaks.
- In **dev with StrictMode**, React mounts → unmounts → mounts again to test your cleanup. This is normal, not a bug.
- Function components with hooks replaced class lifecycle methods.

> 💡 **Memory tip:** Birth (mount) → Life (updates) → Death (unmount, clean up your mess).
>
> 🎤 **Say it in one line:** "Components mount, update and unmount, and useEffect lets me run code at each stage."

1️⃣3️⃣ Styling

> **Four common ways to style React components.**

📖 Explanation

React doesn't force a styling method. Pick based on project size and team preference. All of them work with `className` or `style`.

💻 Code

```
// 1️⃣ Plain CSS file (global): simple, but class names can clash
import "./Card.css";
<div className="card">Hello</div>

// 2️⃣ CSS Modules (scoped): file named Button.module.css
import styles from "./Button.module.css";
<button className={styles.primary}>Click</button>
// Button.module.css →  .primary { background: blue; color: white; }
// Class names become unique automatically (e.g. Button_primary__x7a2)

// 3️⃣ Inline styles: object + camelCase, good for dynamic values
<div style={{ backgroundColor: "tomato", fontSize: "18px" }}>Hi</div>
<div style={{ width: `${progress}%` }}>Progress bar</div>

// 4️⃣ Tailwind CSS: utility classes directly in JSX
<button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700">
  Save
</button>

// Conditional classes (works with any method)
<button className={`btn ${isActive ? "active" : ""}`}>Tab</button>
```

🧠 Comparison

Method	Scoped?	Dynamic styles	Best for
Plain CSS	❌ Global	Via classes	Small projects
CSS Modules	✅ Yes	Via classes	Medium/large apps, avoiding clashes
Inline	✅ Yes	✅ Easy	One-off dynamic values
Tailwind	✅ Yes	Via classes	Fast UI building, consistent design
styled-components	✅ Yes	✅ Props-based	CSS-in-JS lovers

✅ Key Points

- Use `className`, not `class`, to apply CSS classes.
- **Plain CSS** is global, so class names can clash across components.
- **CSS Modules** (`*.module.css`) auto-generate unique class names, so they are scoped.
- Import modules as an object: `styles.primary`.
- **Inline** `style` takes an **object** with **camelCase** keys (`backgroundColor`).
- Inline styles can't do **hover, media queries or pseudo-classes**; use CSS for those.
- **Tailwind** = small utility classes (`p-4`, `flex`, `text-lg`) written directly in JSX.
- **styled-components** = write CSS inside JavaScript as component templates (CSS-in-JS).
- Build **conditional classes** with template literals (or the `clsx` library).
- Put **global styles** (reset, fonts, variables) in `index.css`.

> 💡 **Memory tip:** Plain CSS = shared closet. CSS Modules = your own locked drawer. Tailwind = pre-made outfit pieces.
>
> 🎤 **Say it in one line:** "React supports plain CSS, CSS Modules, inline styles and utility or CSS-in-JS libraries like Tailwind."

📝 Part 1 Cheat Sheet

Topic	Remember this
React	UI library, components + virtual DOM, declarative, one-way data flow
Setup	`npm create vite@latest` → `npm install` → `npm run dev`; entry is `main.jsx`
JSX	One root, close all tags, `className`, camelCase, `{}` for JS expressions
Components	Capitalized function returning JSX; compose and reuse
Props	Read-only, parent → child; destructure; `children`; can pass functions
State	Remembered data; `useState`; setter triggers re-render; never mutate directly
Events	camelCase, pass function (don't call it), `e.preventDefault()`
Conditional	Ternary, `&&` (use real booleans!), early return, `null`
Lists	`map()` + unique stable `key`; avoid index and random keys

Topic	Remember this
Forms	Controlled = `value` + `onChange`; one handler with `[name]: value`
Lifting state	Move state to the closest common parent; data down, callbacks up
Lifecycle	Mount / update / unmount → `useEffect` with `[]`, `[dep]`, cleanup
Styling	CSS, CSS Modules, inline object, Tailwind, styled-components

🔥 Top 5 Quick-Fire Questions

1. **Why do lists need keys?** → So React can identify which items changed, moved or were removed.
2. **Props vs state?** → Props come from the parent and are read-only; state is owned by the component and can change.
3. **Controlled vs uncontrolled?** → Controlled keeps the value in state; uncontrolled keeps it in the DOM, read via ref.
4. **Why can't I call** `onClick={fn()}` **?** → It runs during render. Pass the function itself: `onClick={fn}`.
5. **When do I lift state up?** → When sibling components need to share the same data.

✅ **Part 1 complete.** Next: **Part 2 — Hooks** (Topics 14–23).